



SAPIENZA
UNIVERSITÀ DI ROMA

ML Models for the prediction of gases and their family through SCA sensors

Faculty of Computer Engineering, Informatics and Statistics
Department di Computer Engineering, Automatics and Management
“Antonio Ruberti”
Degree course in Engineering in Computer Science and Artificial
Intelligence

Giorgio Sisti
Number 2134719

Supervisor
Fabio Patrizi

External Supervisor
Roberto Simmarano

A.Y. 2025-2026

ML Models for the prediction of gases and their family through SCA sensors

Student
Giorgio Sisti

Supervisor
Fabio Patrizi

External supervisor
Roberto Simmarano



SAPIENZA
UNIVERSITÀ DI ROMA

Abstract

This thesis aims to design, implement and evaluate Machine Learning models for improving gas classification and gas-family consistency in Sensichips s.r.l. chips used in Smart Cable Air (SCA) sensors. Starting from the company baseline model, based on a Multi-Layer Perceptron (MLP), different classification strategies were investigated. Starting from the company baseline model, several classification strategies were explored, including multi-task, cascade architectures and hierarchical approaches to jointly model gases and their chemical families. Different input representations were also investigated, comparing raw sensor features with statistical features extracted from the temporal signals. The experimental evaluation considers gas accuracy, family accuracy and gas-family consistency. The results highlight the importance of the chosen feature representation and confirm the relevance of the hierarchical structure for ensuring coherent predictions.

Summary

1. INTRODUCTION	5
1.1 GAS DETECTION AND SENSORS	5
1.2 SENSICHIPS S.R.L.	7
1.3 SCA SENSORS	8
1.4 THESIS GOALS AND STEPS	10
2. BACKGROUND	11
2.1 SUPERVISED LEARNING AND CLASSIFICATION MODELS	11
2.2 MULTI-LAYER PERCEPTRON	15
2.3 RANDOM FOREST	19
2.4 MULTI-TASK LEARNING	21
2.5 LOSS FUNCTIONS.....	23
3. DATASET AND PREPROCESSING	27
3.1 SENSICHIPS AIR SENSOR DATASET	27
3.2 GAS AND CHEMICAL FAMILY LABELS	28
3.3 EXPERIMENTAL FEATURES AND NORMALIZATION STRATEGIES	30
3.3.1 Raw features.....	30
3.3.2 Statistical features extraction	30
3.4 CROSS-VALIDATION PROTOCOL.....	31
4. PROPOSED MODELS	33
4.1 INDUSTRIAL BASELINE MODEL	33
4.2 MULTI-TASK AND RANDOM FOREST ARCHITECTURE.....	34
4.2.1 MLP with raw features	34
4.2.2 MLP with statistical feature	35
4.2.3 RF with raw features	36
4.2.4 RF with statistical features.....	37
4.2.5 Window-size model	38
4.3 MULTI-TASK CASCADE ARCHITECTURE	39
4.3.1 Separated Cascade.....	40
4.3.2 Joint Cascade.....	42
4.4 MULTI-TASK ORACLE ARCHITECTURE	45
5. EXPERIMENTAL SETUP	47
5.1 PYTHON ENVIRONMENT AND LIBRARIES	47

5.2 EXPERIMENTAL PIPELINE	49
5.2.1 Data Loading and Label Encoding	49
5.2.2 Feature Preparation and Normalization	50
5.2.3 Train/Validation Split Management	51
5.2.4 Model Training and Prediction Flow	52
5.3 MODEL CONFIGURATION	53
5.3.1 MLP-Based Models	53
5.3.2 Random Forest Models	54
5.3.3 Cascade and Oracle Models	55
5.3.4 Window-Based Models	56
5.5 HYPERPARAMETERS SETTING	57
5.6 LOSS FUNCTION SETTINGS	59
5.7 COMMAND-LINE ARGUMENTS	61
5.8 EVALUATION METRICS	63
5.8.1 Gas Accuracy	63
5.8.2 Family Accuracy	63
5.8.3 Gas-Family Consistency	63
5.8.4 Mean and Standard Deviation Across Folds	64
6. EXPERIMENTAL RESULTS	65
6.1 INDUSTRIAL BASELINE MODEL	65
6.2 EXPERIMENTAL MODELS' RESULTS	68
6.3 EXTERNAL TEST – MODELS' PERFORMANCES COMPARISON	81
6.4 OVERALL COMPARISON	82
7. CONCLUSIONS AND FUTURE WORK	85
8. BIBLIOGRAPHY	86

1. Introduction

Gas sensing technologies play an increasingly important role in several application domains, including environmental monitoring, industrial safety, indoor air-quality assessment, food quality control and medical diagnostics. Their relevance is mainly related to the need for detecting and monitoring hazardous or informative volatile compounds in a reliable, compact and cost-effective way. Air pollution, for instance, represents one of the major environmental risks for human health, and the availability of distributed sensing systems can support continuous monitoring of pollutants and harmful gases [1,2]. Among the different gas sensing technologies, metal-oxide semiconductor sensors are widely used due to their low cost, robustness, compact size, long lifetime and high sensitivity to several target gases [2]. However, gas detection is often complicated by the presence of cross-sensitivity, environmental variability and overlapping sensor responses among different chemical compounds. For this reason, modern gas sensing systems are increasingly combined with signal processing and Machine Learning techniques, which can extract discriminative patterns from sensor responses and improve the identification of gases or gas mixtures [3,4]. In this context, the development of intelligent gas sensing systems is particularly relevant for applications requiring automatic, real-time and embedded classification. This work focuses on Machine Learning models for gas and chemical family classification using data acquired from Smart Cable Air sensors developed by Sensichips s.r.l.

1.1 Gas detection and sensors

Gas leak detection methods became a necessity after the discovery of the effects of harmful gases on human health. Before modern electronic sensors, detection was done using less precise detectors. In the 19th and early 20th centuries coal miners were used to bring canaries in tunnels as a simple detector for gases such as carbon dioxide, carbon monoxide and methane, they simply checked for the canary stop singing or eventually die if not removed from those gases, making the miners signal to exit from there quickly to

survive. The first gas detector in the industrial age was the “Davy lamp” invented by Sir Humphry Davy in England in 1815 to detect the presence of methane in mines. Its working was simple: the variations of the flame height meant the presence of methane, if higher, or the lack of oxygen, if lower [5].

The modern era of gas detection started with catalytic combustion sensor (LEL) by Dr. Oliver Jhonson as researcher for Standard Oil Company in California. The goal of the company was to find a method to detect combustible mixtures in air to prevent explosions in fuel storage tanks, this led to the development of Model A and then Model B “electric vapor indicator” in 1927. Jhonson-Williams Instrument, the first electronics company in Silicon Valley, pioneered many inventions in modern gas detection, making also instruments smaller and more portable. During the following years gas sensors have been incorporated into a wider range of systems such as cars for emissions control, passenger comfort and safety, carbon dioxide sensors in buildings for ventilation systems and many others like gas monitors and alarms for carbon monoxide in domestic places. In the 1990s Metal-Oxide-Semiconductor (MOS) sensors were introduced and became important environmental gas detectors.

These instruments can be classified according to their operating principle and their form factor. From the point of view of the operating mechanism, the main categories include electrochemical sensors, catalytic bead sensors, photoionization detectors, infrared point sensors, infrared imaging sensors, semiconductor sensors, ultrasonic gas leak detectors and holographic sensors.

1. Electrochemical sensors detect gases through oxidation or reduction (redox) reactions at an electrode, generating a current proportional to the gas concentration.
2. Catalytic bead sensors are mainly used for combustible gases and exploit the heat produced by catalytic oxidation.
3. Photoionization detectors use ultraviolet radiation to ionize gas molecules, measuring the resulting current.
4. Infrared sensors detect gases by measuring the absorption of infrared radiation at specific wavelengths.
5. Infrared imaging systems, in addition to the previous ones, can help localize the source of a leak.

6. Semiconductor sensors, commonly based on metal-oxide materials, detect gases through changes in electrical resistance caused by surface chemical reactions.
7. Ultrasonic detectors identify gas leaks by detecting the acoustic emission produced by pressurized gas escaping through an opening.
8. Holographic sensors exploit changes in the optical response of a polymer film containing a hologram.



Figure 1: Gas sensors with different technological systems.

1.2 Sensichips s.r.l.

Sensichips S.r.l. is an Italian company focused on the development of high-performance sensors and sensor solutions for health, safety and environmental monitoring applications. The company works on miniature multi-sensor microsystems, combining heterogeneous sensing materials, microelectronics, signal acquisition and data processing in compact sensing platforms [6].

A central technology developed by Sensichips is SENSIPLUS, a microelectronics platform for the integration of multiple heterogeneous sensors into a single chip or into miniature Multi-Sensor-Microsystems (MSM). The main objective of the platform is to combine heterogeneous functional materials with CMOS microelectronics, obtaining compact microsensors suitable for environmental sensing and other monitoring applications. A key aspect of SENSIPLUS is the cooperative operation of multiple

sensors. The responses acquired from different sensing elements can be jointly processed through Artificial Intelligence and analytical algorithms to improve detection and classification capabilities. In this sense, it provides hardware support for smart sensing systems, where sensing, acquisition and data processing are integrated into compact architecture.

1.3 SCA sensors

The Smart Cable Air (SCA) device is a system-in-package gas sensing platform that integrates two main components: the SENSIMOX gas sensor and the SENSIPLUS ASIC. SENSIMOX is an application-specific MEMS-based (Micro-Electro-Mechanical Systems) metal oxide semiconductor (MOX) gas sensor, while SENSIPLUS is a CMOS (Complementary Metal-Oxide-Semiconductor) application-specific integrated circuit designed to perform AC impedance spectroscopy measurements and sensor signal acquisition.

- The SENSIMOX device is based on MEMS hotplates and MOX sensing layers. Its structure includes a heater, operated in temperature cycled operation (TCO), and a sensing layer read using an AC impedance readout scheme. The device was designed for compact integration with the SENSIPLUS ASIC and includes two sensors, with separated etch-pits (geometric micro-depressions on the surface of a crystalline material when exposed to a chemical etchant) to reduce thermal crosstalk (). The sensing layers are based on nanostructured SnO₂, with one layer decorated with Au clusters.
- The SENSIPLUS chip controls the SENSIMOX heater and gets the sensor response. The heater temperature is sequentially modulated from 150 °C to 400 °C in 50 °C steps. At each temperature, the impedance of the sensitive film is measured at two frequencies, 200 Hz and 78.125 kHz, acquiring the in-phase component of the sensor response. This configuration produces 12 virtual MOX sensor signals.

In addition to the external SENSIMOX sensor, SENSIPLUS integrates on-chip aluminum interdigitated gas detectors. These sensors work using voltage cycling

operation (VCO) while acquiring the AC impedance. The selected readout configuration uses a fixed frequency of 78.125 kHz, two voltage magnitudes, 400 mV and 3 V peak-to-peak, and both in-phase and quadrature components. This generates 4 additional virtual sensor signals. Together with the 12 SENSIMOX signals, the SCA device therefore gets a total of 16 sensor features. All sensor signals are acquired through the reconfigurable analogue blocks of the SENSIPLUS ASIC, which allow programmable stimulation of the sensors and high-resolution acquisition of their responses. The complete SCA system-in-package has a size of $5.5 \times 3.9 \times 3 \text{ mm}^3$, weighs 365 mg and consumes an average of 2.5 mW while reading one sample of 16 features every 1.04 s.

Multiple SCA devices, for distributed sensing applications, can be connected through the proprietary SENSIBUS communication protocol, using a cable bus with power, ground and a single data line and up to 256 SCA devices can be mounted on a cable bus up to 150 m long, making the platform suitable for distributed chemical sensing in safety and security monitoring scenarios.

The combination of impedance spectroscopy, temperature cycled operation and multiple virtual sensor signals increases the dimensionality of the sensor response, so that the 16 features are then used as input for machine learning algorithms for gas classification.



Figure 2: SCA air sensor.

1.4 Thesis goals and steps

This work of thesis aims at designing, implementing and evaluating machine learning models for gas classification using data acquired from Smart Cable Air sensors.

The starting point is the sensing architecture described in the previous section, having the 16 raw features provided by the SCA device through the combination of SENSIMOX and SENSIPLUS on-chip sensors. From this reference, different modelling strategies are investigated to evaluate their effect on gas recognition accuracy and gas-family consistency, moreover, both raw sensor features and statistical features extracted from the temporal evolution of the sensor signals are considered.

The first step consists in extending the classification task from gas prediction only to a multi-task setting, where the model predicts both the gas and the corresponding chemical family. This allows the model to exploit the hierarchical relation, through configurable loss functions, between gases and chemical families and makes it possible to evaluate not only the correctness of the gas prediction, but also the chemical coherence of the output.

A second step consists in developing cascade architectures, in which the prediction of the chemical family is used as additional information for the gas classifier. Both separated and joint cascade models are considered, with hard and soft representations of the predicted family. In addition, an oracle architecture is introduced to evaluate the behavior of the gas classifier when the correct family information is provided a priori.

Finally, considering statistical descriptors extracted from the whole files or temporal sliding windows was performed an experimental evaluation comparing gas and family accuracy together with gas-family consistency between different configurations.

2. Background

In this chapter we will focus on the theoretical aspects used during the design and evaluation phases of this work. Since the goal is the classification of gases and chemical families from sensor data, the first part focuses on supervised learning and multi class classification, which provides the general framework for learning a mapping between input sensor features and target labels. Main models used in the experimental analysis are presented such as Multi-Layer Perceptron, as feedforward neural network able to learn non-linear relations between input features and output classes, and Random Forest, used for its ability of aggregation of multiple decision trees prone to classification and regression tasks. Finally multi-task learning and loss functions are discussed having the most important role in the experimental part. Indeed, relations like gas and chemical family predictions are treated as related tasks and able of affecting their own accuracy and chemical consistency.

2.1 Supervised learning and classification models

Machine Learning can be broadly defined as the study of algorithms that allow computers to learn patterns from data and improve their performance on a given task without being explicitly programmed for every possible case. Depending on the type of data available and on the desired output, Machine Learning methods are commonly divided into supervised learning, unsupervised learning, semi-supervised learning and reinforcement learning [9].

In supervised learning, the training dataset is composed of input-output pairs. Each sample is represented by an input vector (x), containing the observed features, and by a target value (y), which represents the desired output.

The goal is to learn a function:

$$f: X \rightarrow Y$$

able to map each input sample to the corresponding output. Given a training set:

$$D = \{(x_i, y_i)\}_{i=1}^N$$

the model learns its parameters by minimizing a loss function that measures the difference between the predicted output ($\hat{y}_i = f(x_i)$) and the true label (y_i).

In unsupervised learning, the objective is to discover hidden structures, similarities or clusters inside the data without predefined target labels.

Semi-supervised learning combines labeled and unlabeled data, while reinforcement learning is based on an agent that interacts with an environment and learns a policy by receiving rewards or penalties after its actions.

Supervised learning problems are usually divided into two main categories: classification and regression. In classification, the target variable belongs to a finite set of discrete classes:

$$y \in \{1, 2, \dots, C\}$$

and the model assigns each input to one of these categories.

In regression, the target variable is continuous:

$$y \in \mathbb{R}$$

and the model estimates a numerical value. Therefore, classification is mainly used when the objective is to recognize categories, labels or classes, while regression is used for numerical prediction problems.

The general supervised learning objective can be written as an empirical risk minimization problem:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i)$$

where (θ) represents the model parameters and (L) is the loss function. Different supervised learning algorithms differ mainly in the form of the function (f_{θ}) , in the assumptions made about the data, and in the optimization procedure used to estimate the parameters.

Several families of supervised learning methods exist. Linear regression models continuous target variables as a linear combination of the input features:

$$\hat{y} = \beta_0 + \sum_{j=1}^d \beta_j x_j$$

and is mainly used in regression analysis and numerical prediction problems.

Logistic regression is used for classification and estimates the probability of a class through the sigmoid function:

$$P(y = 1|x) = \sigma(\theta^T x) = \frac{1}{(1 + e^{(-\theta^T x)})}$$

It is commonly applied to binary classification problems and probabilistic decision-making tasks.

Naive Bayes is a probabilistic supervised learning method based on Bayes' theorem and on the conditional independence assumption among features. Its rule can be written as:

$$\hat{c} = \arg \max_c P(c) \prod_{j=1}^d P(x_j|c)$$

This type of model is mainly used in probabilistic classification problems, especially when the input features can be modelled through class-conditional probability distributions.

Decision trees represent supervised models based on a recursive partition of the input space. Each internal node applies a test on one feature, while each leaf assigns a class or a prediction value. A generic decision rule can be expressed as:

$$x_j \leq t$$

where (x_j) is a feature and (t) is a threshold. Decision trees are widely used when interpretability is important, since the final prediction can be represented as a sequence of human-readable rules.

Instance-based methods, such as k-Nearest Neighbors, classify a new sample according to the labels of the closest training samples. The prediction rule can be written as:

$$\hat{y} = \text{mode } y_i: x_i \in N_k(x)$$

where $(N_k(x))$ is the set of the (k) nearest neighbours of (x) . These methods are mainly used in pattern recognition and non-parametric classification.

Support Vector Machines are margin-based classifiers. In the linear binary case, the objective is to find a separating hyperplane:

$$w^T x + b = 0$$

that maximizes the margin between classes. A common soft-margin formulation is:

$$\min_{w,b,\xi} \frac{1}{2} |w|^2 + C \sum_{i=1}^N \xi_i$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \xi_i \geq 0$$

Support Vector Machines are mainly used in problems where margin maximization and good generalization are important.

Neural networks, including Multi-Layer Perceptron, model the relationship between input and output through multiple layers of weighted transformations and non-linear activation functions. A feedforward neural network can be generally written as:

$$f(x) = W_L \phi(W_{L-1} \phi(\dots \phi(W_1 x + b_1) \dots) + b_{L-1}) + b_L$$

where (W_l) and (b_l) are the weights and biases of layer (l) , and (ϕ) is a non-linear activation function. Neural networks are mainly used when the relationship between inputs and outputs is complex and non-linear.

In summary, supervised learning provides the theoretical framework for learning predictive models from labeled data. Its main objective is to generalize from training examples to unseen samples. Within this framework, classification methods are particularly relevant when the output belongs to a finite set of classes, while regression methods are used when the target is continuous [7].

2.2 Multi-Layer Perceptron

MLP is a feedforward neural network represented as a finite directed acyclic graph in which information flows from the input to the output layers through one or more hidden ones. The nodes in this graph are called neurons. Neurons that are no target of any connection are called input neurons and for an input pattern of dimension n there must be n input neurons, while those which are no source of any connection are called output neurons, they can be multiple and their number depends on how the desired values patterns are described, finally those which are not input nor output are called hidden neurons. The acyclicity of this kind of structure make possible to all neurons be organized in layers (first layer = input layer, central layers = hidden layers, last layer = output layer).

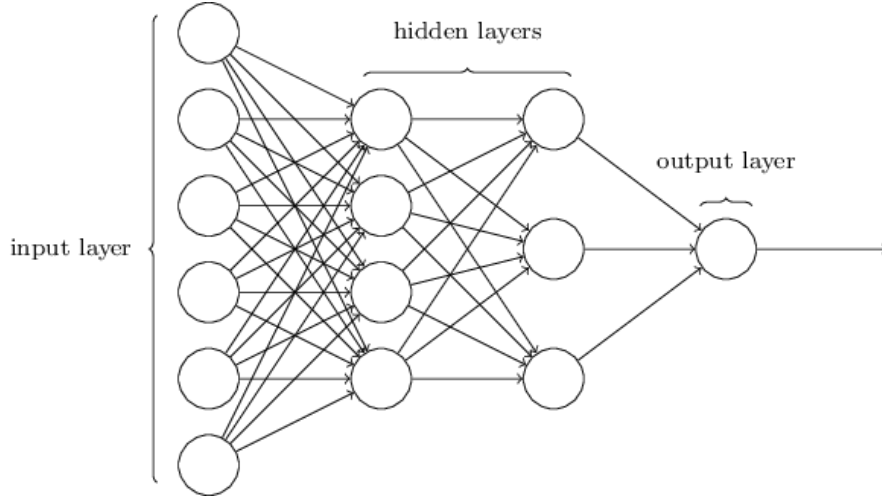


Figure 3: MLP architecture.

All nodes are enumerated and relations between them exist:

- $Succ(i)$ is the set of all neurons j for which a connection $i \rightarrow j$
- $Pred(i)$ is the set of all neurons j for which a connection $j \rightarrow i$

All connections are weighed with a real number and have a bias, the weight for $i \rightarrow j$ connection is named w_{ji} and the bias weight is named w_{i0} .

For each hidden or output neuron (i), the input signal is computed as:

$$net_i = w_{i0} + \sum_{j \in Pred(i)} w_{ij} a_j, \quad a_j = \text{activation of neuron } j$$

The activation of neuron (i) is then obtained by applying an activation function:

$$a_i = f_{log}(net_i)$$

Common activation functions include the logistic sigmoid

$$f_{log}(net_i) = \frac{1}{1 + e^{-net_i}}$$

the hyperbolic tangent

$$f_{tanh}(net_i) = \tanh(net_i)$$

and the linear function:

$$f_{lin}(net_i) = net_i$$

During training, the goal is to adapt the weight vector of the network in order to minimize an error function defined over the training dataset. Given a dataset (D) composed of (p) input-output pairs, the error can be written as:

$$E(\vec{w}; D) = \frac{1}{2} \sum_{k=1}^p \|y(\vec{x}^{(k)}; \vec{w}) - \vec{d}^{(k)}\|^2$$

where $(y(\vec{x}^{(k)}; \vec{w}))$ is the network output for the input pattern $(x^{(k)})$, $(d^{(k)})$ is the desired output, and $(\vec{w})$ represents the network weights. Learning therefore corresponds to finding the weights that minimize this error function.

The optimization process is usually based on gradient-based methods. In the simplest case, gradient descent updates the weight vector in the opposite direction of the gradient of the error function:

$$w \leftarrow w - \varepsilon \nabla E(w)$$

where (ε) is the learning rate. A small learning rate generally leads to more stable convergence, while a large learning rate may cause divergence. However, very small learning rates can make training slow.

The gradients required for the weight update are computed through backpropagation. Backpropagation applies the chain rule to propagate the error from the output neurons backwards through the network. For a weight connecting neuron (j) to neuron (i), the partial derivative of the error can be written as:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial net_i} \frac{\partial net_i}{\partial w_{ij}}$$

Since:

$$net_i = w_{i0} + \sum_{j \in Pred(i)} w_{ij} a_j$$

it follows that:

$$\frac{\partial net_i}{\partial w_{ij}} = a_j$$

and therefore:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial net_i} a_j$$

The same principle is applied recursively from the output layer to the input layer, allowing the computation of the derivatives for all weights in the network.

Training is performed iteratively over the training set. For each epoch, the input patterns are propagated forward through the network, the error is computed, gradients are calculated through backpropagation, and the weights are updated according to the chosen optimization method. After training, the model parameters are fixed.

During testing, the MLP does not update its weights. Each unseen input sample is only propagated forward through the network, and the final output activations are used to produce the prediction. Therefore, the training phase is responsible for learning the model parameters, while the testing phase evaluates the ability of the trained network to generalize to new data [8][9].

Algorithm A Generic MLP Learning Algorithm

Require: Training dataset $\mathcal{D} = \{(\vec{x}_i, \vec{d}_i)\}_{i=1}^N$, error function E , learning/minimization approach

Ensure: Optimized weight vector $\vec{w}$

- 1: Choose an initial weight vector $\vec{w}$
- 2: Initialize the minimization approach
- 3: **while** the error has not converged **do**
- 4: **for** each training pattern $(\vec{x}, \vec{d}) \in \mathcal{D}$ **do**
- 5: Apply $\vec{x}$ to the network and compute the network output
- 6: Compute the partial derivatives for all weights:

$$\frac{\partial e(\vec{x})}{\partial w_{ij}}$$

- 7: **end for**
- 8: Compute the total gradient over all training patterns:

$$\frac{\partial E(\mathcal{D})}{\partial w_{ij}}$$

- 9: Perform one update step of the minimization approach
 - 10: **end while**
 - 11: **return** optimized weight vector $\vec{w}$
-

2.3 Random Forest

Random Forest is a supervised ensemble learning method based on the combination of multiple decision trees. Instead of relying on a single tree, the model builds a collection of trees and aggregates their predictions in order to obtain a more stable and accurate final output. It can be used both for classification and regression, but in this thesis the focus is on its use as a classification algorithm. The main idea behind this model is to introduce randomness during the construction of the trees. Each tree is trained on a different bootstrap sample extracted from the original training set, and at each split only a random subset of the input features is considered. This strategy reduces the correlation between individual trees and improves the generalization capability of the ensemble.

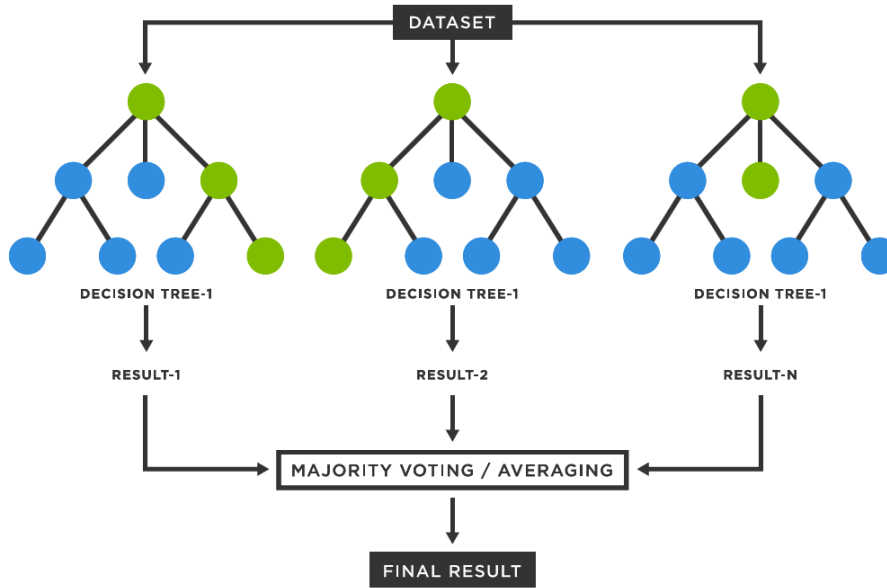


Figure 4: Random Forest architecture.

Given a training set:

$$D = (x_i, y_i)_{i=1}^N$$

A Random Forest composed of (B) trees generates (B) different bootstrap datasets:

$$D_1, D_2, \dots, D_B$$

Where each (D_b) is obtained by sampling with replacement from (D) . For each bootstrap dataset, a decision tree $(h_b(x))$ is trained. During the construction of each tree, the algorithm recursively partitions the input space. At each node, a subset of features is randomly selected, and the best split is chosen by minimizing an impurity criterion. A candidate split divides the node into two subsets, (S_L) and (S_R) , and the selected split is the one that minimizes the weighed impurity:

$$\Delta I = \frac{|S_L|}{|S|} I(S_L) + \frac{|S_R|}{|S|} I(S_R)$$

where $(I(\cdot))$ represents the chosen impurity measure.

During training, each tree is grown independently using its bootstrap dataset and its random feature selections.

The final forest is therefore composed of many diverse decision trees:

$$RF(x) = h_1(x), h_2(x), \dots, h_B(x)$$

During testing, the parameters and structure of the trees are fixed. A new input sample is passed through all trees of the forest, and each tree produces a class prediction. In classification, the final prediction is obtained by majority voting:

$$\hat{y} = \text{mode } h_1(x), h_2(x), \dots, h_B(x)$$

Equivalently, the predicted class can be written as:

$$\hat{y} = \arg \max_c \sum_{b=1}^B \mathbb{I}(h_b(x) = c)$$

where $(\mathbb{I}(\cdot))$ is an indicator function equal to 1 when tree (b) predicts class (c), and 0 otherwise.

The use of many randomized trees makes Random Forest less sensitive to the instability of individual decision trees. A single decision tree can easily overfit the training data, especially when it is very deep. By averaging or voting over many trees trained on different bootstrap samples and feature subsets, this model usually achieves better

generalization and reduces overfitting. However, this improvement comes at the cost of lower interpretability compared to a single decision tree, since the final decision depends on the combined behavior of many models [10][11].

Algorithm B Random Forest Classification Algorithm

Require: Training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, number of trees B , number of randomly selected features m , impurity criterion I , test sample x

Ensure: Predicted class $\hat{y}$ for x

```

1: for  $b = 1, \dots, B$  do
2:   Draw a bootstrap sample  $\mathcal{D}_b$  from  $\mathcal{D}$ 
3:   Train a decision tree  $h_b$  on  $\mathcal{D}_b$ 
4:   while the current node is not terminal do
5:     Randomly select  $m$  features from the full feature set
6:     Evaluate candidate splits using only the selected  $m$  features
7:     Choose the split that minimizes the node impurity according to  $I$ 
8:     Split the node and continue recursively
9:   end while
10: end for
11: for each tree  $h_b$  do
12:   Compute the class prediction:

```

$$c_b = h_b(x)$$

```

13: end for
14: Return the final prediction by majority voting:

```

$$\hat{y} = \text{mode}\{c_1, c_2, \dots, c_B\}$$

2.4 Multi-task learning

Multi-Task Learning aims to leverage useful information contained in multiple learning tasks to help learn a more accurate learner for each task. Given (m) learning tasks:

$$\{T_i\}_{i=1}^m$$

based on an assumption that all the tasks, or at least a subset of them, are related, jointly learning multiple tasks is empirically and theoretically found to lead to better performance than learning them independently. Based on the nature of the tasks, MTL can be classified into several settings, including multi-task supervised learning, multi-task unsupervised learning, multi-task semi-supervised learning, multi-task active learning, multi-task reinforcement learning, and multi-task online learning. In multi-task supervised learning, each task, which can be a classification or regression problem, is to predict labels for unseen data instances given a training dataset consisting of training data instances and

their labels. The basic assumption is that related tasks may share useful information, and that this shared information can help the model learn more robust and general representations. In supervised MTL, each task is associated with a labeled dataset:

$$\mathcal{D}_i = (x_j^i, y_j^i)_{j=1}^{n_i}$$

where (x_j^i) is the input sample of task $(\mathcal{T}_i)$, and (y_j^i) is its corresponding label. If (y_j^i) is discrete, the task is a classification problem; if it is continuous, the task is a regression one. This architecture aims to learn a set of functions:

$$f_i(x)_{i=1}^m$$

one for each task, such that each function (f_i) can correctly predict the output of unseen samples belonging to task $(\mathcal{T}_i)$. Differently from single-task learning, where each model is trained separately, MTL trains the tasks together and introduces a mechanism for sharing information among them.

A common neural implementation is based on shared hidden layers and task-specific output layers. The shared part of the network learns a common representation of the input:

$$h = \phi(W_s x + b_s)$$

where (W_s) and (b_s) are shared parameters and (ϕ) is a non-linear activation function. Then, each task has its own output head:

$$\hat{y}_i = g_i(W_i h + b_i)$$

where (W_i) and (b_i) are task-specific parameters, and (g_i) is the output function associated with task $(\mathcal{T}_i)$. The training objective is usually expressed as a weighted combination of the losses of all tasks:

$$\mathcal{L}_{MTL} = \sum_{i=1}^m \lambda_i \mathcal{L}_i(\hat{y}_i, y_i)$$

where $(\mathcal{L}_i)$ is the loss of task (i), and (λ_i) controls the contribution of that task to the total objective. During training, the shared parameters are updated using the information coming from all tasks, while the task-specific parameters are optimized for their corresponding outputs.

MTL models can share information in different ways:

- Feature-based approaches assume that different tasks share a common feature representation.
- Parameter-based approaches encode task relatedness through constraints or regularization on model parameters.
- Instance-based approaches use data instances from different tasks to improve the learning process, often by assigning different weights to samples coming from related tasks.

Multi-Task Learning is particularly useful when the amount of data available for each task is limited and related between them, making the shared information act as an inductive bias, improving generalization. This characteristic has made MLT been applied in several research areas, including computer vision, bioinformatics, speech processing, natural language processing and ubiquitous computing [12].

2.5 Loss Functions

A loss function is a mathematical function used to measure the discrepancy between the output predicted by a model and the true target value. During training, the learning algorithm adjusts the model parameters in order to minimize this discrepancy. Therefore, the loss function defines the optimization objective of the learning process and has a direct influence on the behavior and performance of the classifier

Given a training dataset:

$$D = (x_i, y_i)_{i=1}^N$$

and a classifier (f_θ), where (θ) represents the trainable parameters, the general learning objective can be written as:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_\theta(x_i), y_i)$$

where (L) is the loss function, $(f_{\theta}(x_i))$ is the prediction of the model and (y_i) is the true class label. Loss functions for classification can be organized according to different criteria. The first distinction concerns the type of measurement used to compute the loss:

- Distance-based loss functions, the loss depends on the distance between the predicted value and the true value. The larger this distance is, the larger the assigned loss becomes.
- Direction-based loss functions, instead, the loss is related to the direction or alignment between the prediction and the target.

Second distinction concerns the shape of the loss function:

- Continuous loss functions assign values that change smoothly over the prediction space.
- Semi-continuous loss functions include regions in which the loss is constant, often equal to zero, and regions in which it changes continuously.
- Discrete loss functions assign only a finite set of values, typically distinguishing between correct and incorrect predictions.

Finally, the third distinction concerns the mathematical form of the loss function:

- Linear loss functions assign penalties that grow linearly with the error.
- Non-linear loss functions assign penalties according to nonlinear transformations, such as squared or logarithmic functions.

The most common distance-based loss functions are the Absolute Error Function, that measures the absolute difference between the true value and the predicted value:

$$L_{AE}(y, \hat{y}) = |y - \hat{y}|$$

The Squared Error Function that penalizes the squared difference between prediction and target giving stronger penalties to large errors and being more sensitive to outliers than the previous one:

$$L_{SE}(y, \hat{y}) = (y - \hat{y})^2$$

The Huber loss combines the ones described above ensuring robustness to outliers when required.

For classification tasks, one of the most widely used losses is cross-entropy. Cross-entropy measures the difference between the true class distribution and the predicted probability distribution. In binary classification, it is commonly written as:

$$L_{BCE}(y, \hat{p}) = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

where $(y \in 0,1)$ is the true label and $(\hat{p})$ is the predicted probability of the positive class. Binary cross-entropy is therefore suitable for two-class classification problems.

For multi-class classification, the corresponding formulation is softmax cross-entropy:

$$L_{CE}(y, \hat{p}) = - \sum_{c=1}^C y_c \log(\hat{p}_c)$$

where (C) is the number of classes, (y_c) is the one-hot encoded target and $(\hat{p}_c)$ is the predicted probability for class (c) . This loss is particularly suitable when each sample belongs to exactly one class among several possible classes.

When the dataset is imbalanced, standard cross-entropy may give excessive importance to the most frequent classes. In these cases, weighted cross-entropy can be used by assigning a weight to each class:

$$L_{WCE}(y, \hat{p}) = - \sum_{c=1}^C w_c y_c \log(\hat{p}_c)$$

where (w_c) is the weight associated with class (c) . This formulation is useful when minority classes must contribute more strongly to the optimization process.

Focal loss is another loss function designed for difficult or imbalanced classification problems. It modifies cross-entropy by reducing the contribution of easy examples and increasing the relative importance of hard examples:

$$L_{Focal} = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

where (p_t) is the predicted probability of the correct class, (α_t) is a weighting factor and (γ) is a focusing parameter. When (p_t) is high, the sample is already correctly classified with high confidence and its contribution to the loss is reduced. When (p_t) is low, the loss remains high and the model focuses more on that difficult sample.

Semi-continuous losses include the (ε) -insensitive family. These functions do not penalize errors that fall inside a tolerance interval around the true value, they are useful when small deviations are acceptable and should not influence the learning process. Other

types of loss functions are the discrete ones including the Zero-One Loss, which directly measures whether a prediction is correct or incorrect. It works the same on classification accuracy, because it assigns no penalty to correct predictions and a fixed penalty to wrong ones, however since it is discrete and not differentiable, it is generally not convenient for gradient-based optimization in neural networks, where other differentiable losses are usually preferred like the ones previously described.

Choosing a loss means understanding the true structure of the problem so that the most compatible one can be chosen, but generally:

- Regression problems rely on distance-based continuous losses such as absolute error, squared error or Huber loss.
- Binary classification commonly uses sigmoid or binary cross-entropy.
- Multi-class classification typically uses softmax cross-entropy.
- Imbalanced classification can benefit from weighted cross-entropy or focal loss.
- Problems in which small errors are acceptable can use (ϵ) -insensitive losses.
- Direct classification evaluation is represented by zero-one loss.

For this thesis the main functions considered are cross-entropy, focal and hierarchical losses, each one compatible with the multi-class classification problem but also able to increase attention on different aspects as already described [13][14].

3. Dataset and Preprocessing

In this chapter the source dataset is described and all the preprocessing steps done before the model training. The first part introduces the structure of the Sensichips Air Sensor dataset, focusing on the organization of the available measurements and their association with the corresponding gas and family labels. Following, original raw sensor features and the extraction of the statistical descriptors from the temporal signals are explained together with the normalization strategies and the cross-validation protocol used for the experimental phase.

3.1 Sensichips air sensor dataset

As already mentioned, the dataset used in this work was acquired with the Sensichips SCA device and it contains values saved in CSV format, each file represents a complete acquisition of the sensor during a measurement. These files are organized in two parts: the training set and the test set.

Training and test sets are composed of data which is intended to be used in development and validation of Machine Learning models and they are completely public and free to use. For each chemical compound there are multiple acquisitions providing several CSV files for each gas class allowing the model to learn under different acquisition conditions.

The training folder contains company's MLP architecture and model description, together with raw, normalized and one-filtered data.

- Raw data corresponds to values directly acquired from the sensor through its internal analogue-to-digital converter.
- Normalized data contain the same acquisitions of the raw ones with the initial transient phase removed i.e. the part of the signal that corresponds to the adaptation of the sensor response when the device passes from the reference condition, air presence, to the exposure to the target gas. This leads the model to focus on sensor response after the initial adaptation phase reducing the influence of the air.

- One-class-filtered representation corresponds to an additional preprocessing level applied to the original acquisitions, producing a more selected subset of samples for each gas class.

The experiments were performed by exposing the SCA sensor to chemical compounds in liquid form and allowing them to evaporate. Some acquisitions were carried out in an enclosed environment, where the compound slowly evaporated with limited turbulence. This condition allowed the sensor to acquire samples over a gradual concentration variation, from low to higher concentrations.

The test directory instead contains data acquired under less controlled conditions being useful to evaluate the behavior of trained models on acquisitions that are more challenging than the controlled training ones.

3.2 Gas and chemical family labels

In supervised classification, labels represent the target information associated with each input sample. In a multi-class problem, each sample belongs to one class among a finite set of possible categories. Formally, given an input sample (x_i), the corresponding label can be written as:

$$y_i \in 1, 2, \dots, C$$

where (C) is the number of available classes. The objective of the model is to learn a function able to assign each input sample to the correct class. Here we have a single-label multi-class classification problem that considers two different levels: the first is the gas label, which specifies the chemical compound associated with the acquisition, the second level is the chemical family label, which groups gases on their chemical category. The relation between gas labels and family labels is hierarchical. A gas belongs to only one chemical family, while the same chemical family can contain multiple gases.

This defines a many-to-one relation from gas classes to family classes: $g_i \rightarrow f_i$ where (g_i) is the gas label and (f_i) is the corresponding chemical family label. During preprocessing,

gas labels are extracted from the label column, and the corresponding family labels are then obtained through a gas-to-family mapping function.

Gas-to-Family Mapping

25 gas classes and corresponding chemical families

Gas label	Chemical family
1. acetic_acid	acid
2. formic_acid	acid
3. phosphoric_acid	acid
4. apple_vinegar	acid_mixture
5. balsamic_vinegar	acid_mixture
6. ammonia	base
7. sodium_hydroxide	base
8. ammonium_chloride	salt
9. calcium_nitrate	salt_oxidizer
10. ethanol	alcohol
11. bioethanol	alcohol
12. isopropanol	alcohol
13. red_wine	alcohol_mixture
14. acetone	ketone
15. methane	hydrocarbon
16. butane	hydrocarbon
17. gasoline	hydrocarbon_mixture
18. diesel	hydrocarbon_mixture
19. kerosene	hydrocarbon_mixture
20. lighter_fluid	hydrocarbon_mixture
21. hydrogen_peroxide	oxidizer
22. urea	amide
23. air	background
24. water_vapor	background
25. nitromethane	nitro_compound

Figure 5: Gas-to-Family Mapping.

Before being used by the models, both gas and family labels are encoded into numerical values, since Machine Learning algorithms require numerical target representations during training. Those numbers are then used to compute the loss function computation comparing the predicted label with the true one for gases and families. During testing they are not used to update the model parameters anymore, but they consist in the ground truth to evaluate the predictions produced during training. In addition, true and predicted labels are necessary for gas and chemical family accuracy computation and also for their consistency criterion, which are described accurately in the fifth chapter.

3.3 Experimental features and normalization strategies

The experimental analysis considers two main input representations: raw features and statistical features. These representations are used to evaluate whether the models perform better when trained directly on the original sensor outputs or on compact descriptors extracted from the temporal evolution of the signals.

3.3.1 Raw features

The experimental analysis considers two main input representations: raw features and statistical features. These representations are used to evaluate whether the models perform better when trained directly on the original sensor outputs or on compact descriptors extracted from the temporal evolution of the signals.

3.3.2 Statistical features extraction

Statistical features are extracted from the temporal signals contained in each CSV file. Instead of using each sensor readout independently, the signal is summarized through a set of descriptive statistics computed either on the whole acquisition or on temporal windows. This representation provides a compact description of the signal behavior over time and allows the models to exploit information related to the distribution, variation and trend of the sensor response.

Normalization strategies are also applied to make the feature values comparable across dimensions: standardization and min-max normalization are considered. Standardization rescales each feature using the mean and standard deviation computed on the training set:

$$x' = \frac{x - \mu}{\sigma}$$

while min-max normalization maps the values into a fixed numerical range of $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Summary of Statistical Features

Statistical feature	Description
1. Mean	Average value of the signal over time.
2. Standard deviation	Measure of signal variability around the mean.
3. Minimum	Lowest observed value in the signal.
4. Maximum	Highest observed value in the signal.
5. Median	Central value of the ordered signal.
6. Variance	Squared measure of dispersion of the signal.
7. Range	Difference between maximum and minimum values.
8. Interquartile range (IQR)	Difference between the 75th and 25th percentiles.
9. First quartile (Q1)	Value below which 25% of the data fall.
10. Third quartile (Q3)	Value below which 75% of the data fall.
11. Skewness	Measure of asymmetry of the signal distribution.
12. Kurtosis	Measure of the peakedness of the distribution.
13. Root mean square (RMS)	Quadratic mean of the signal values.
14. Mean absolute value (MAV)	Average of absolute signal values.
15. Energy	Sum of squared signal values over time.
16. Entropy	Measure of signal randomness or complexity.
17. Zero-crossing rate	Number of times the signal crosses zero.
18. Peak-to-peak amplitude	Difference between maximum and minimum peaks.
19. Signal slope	Average rate of change over time.
20. Maximum slope	Highest rate of change between consecutive samples.
21. Minimum slope	Lowest rate of change between consecutive samples.
22. Autocorrelation (lag 1)	Correlation of the signal with itself at lag 1.
23. Signal duration	Length of the acquisition window.
24. Area under the curve (AUC)	Integral of the signal over time.

Figure 6: Statistical features for the temporal analysis of the signals.

3.4 Cross-validation protocol

Cross-validation is used to evaluate how well a Machine Learning model generalizes unseen data. In classification, the goal is not only to obtain high accuracy on the training set, but to estimate the expected performance of the learned model on new samples. For this reason, the dataset must be divided into independent training and test subsets, so that the model is trained on one portion of the data and evaluated on samples that were not

used during parameter estimation. Cross-validation techniques are employed to mitigate the problem of overfitting, which occurs when a model is excessively complex and captures the noise along with the underlying data pattern. By partitioning the data into training and testing sets, cross-validation helps in achieving a balance between bias and variance, leading to more robust model performance. In k-fold cross validation, the dataset (D) is divided into (k) disjoint subsets: $D = S_1, S_2, \dots, S_k$, for each iteration (i) one subset (S_i) is used as the test set, while the remaining ones are used as the training set: $T_i = D - S_i$ a model (h_i) is trained on (T_i) and evaluated on (S_i). The final cross-validation error is obtained by averaging errors over all folds:

$$error_{cv} = \frac{1}{k} \sum_{i=1}^k error_{S_i}(h_i)$$

With the corresponding accuracy being:

$$accuracy_{cv} = 1 - error_{cv}$$

In particular, in this thesis a 10-fold cross validation protocol is adopted, the dataset is divided into ten folds, based on predefined splits provided by the company in the file *cv10_splits_train_valid.json*. The JSON file was generated by assigning, for each fold, approximately one validation file for every ten training files, having for each split an approximate 1:10 validation-to-training file ratio. Each fold defines a specific separation between training and validation data, for each iteration, one fold is used as validation set while the remaining nine ones are used for training, resulting in ten independent evaluations for each model configuration. This approach ensures full reproducibility of the experiments and guarantees that all models are evaluated on exactly the same data partitions [9][15].

4. Proposed models

In this fourth chapter all implemented and tested models are described to better understand the different classification strategies used for the gas and family predictions. Starting from the industrial baseline model, extensions and modifications are then introduced including multi-task architectures, cascade models and a theoretical oracle-based configuration. The empirical results of those models are then described in the sixth chapter.

4.1 Industrial baseline model

The industrial baseline model, as already mentioned before, is a Multi-Layer Perceptron (MLP), trained to perform supervised gas classification from raw sensor features. Its initial formulation was a relatively small MLP architecture with three hidden layers composed of 25, 50 and 25 neurons respectively, providing the original reference structure for the classification pipeline. However, the model effectively considered in the experimental work was then updated to a larger MLP architecture composed of two hidden layers with 128 and 64 neurons, having ReLU (Rectified Linear Unit) as activation function and Adam as optimization algorithm with a maximum number of 400 iterations and a fixed random seed equal to 42. Before training, z-score normalization is done so that input features are rescaled to have null mean and unit standard deviation.

This architecture uses only the raw sensor features as input, each CSV row is treated as an independent sample corresponding to a feature vector passed directly to the MLP without any transformation. Indeed, operating at the single-readout level it learns the mapping between sensors acquisitions and gas classes.

Industrial Baseline Model

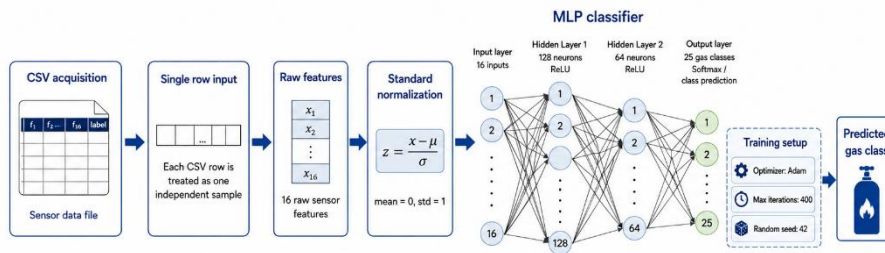


Figure 7: Industrial baseline model architecture, from data preprocessing to class prediction.

4.2 Multi-task and Random Forest architecture

This second paragraph exposes the first modifications and adjustments starting from the model just described above. Multi-Layer Perceptron and Random Forest new implementations are presented with all their characteristics useful to understand the practical differences between those systems. Multi-task learning replaces the single-task used previously, reformulating the problem, considering two related outputs: gas and the corresponding chemical family labels. The purpose of this architecture is to exploit the hierarchical relation between gases and families during training. Since each gas belongs to a specific chemical family, the additional family prediction task provides complementary information that can guide the learning process and help the model produce more coherent predictions. In the following sections, the multi-task and Random Forest approaches are analyzed with different input representations and model configurations, including raw features, statistical features and window-based descriptors.

4.2.1 MLP with raw features

The first multi-task model extends the industrial baseline by modifying the output structure of the MLP. Instead of producing only gas prediction, the model receives the raw sensor features as input and produces two outputs: one for the gas class and the other one for the chemical family class. The input used is a single sensor readout represented by a raw feature vector:

$$x = [x_1, x_2, \dots, x_{16}]$$

This vector is passed through the shared hidden layers of the MLP, which learn a common internal representation of the sensor response. From this shared representation, two separate output branches are generated: the first branch predicts the gas label, while the second branch predicts the chemical family label.

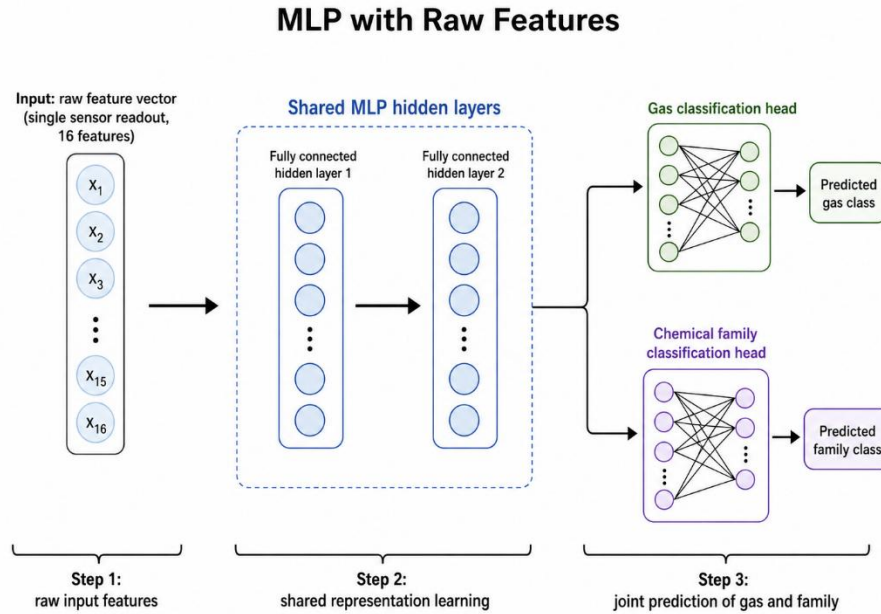


Figure 8: MLP model with raw sensor features.

4.2.2 MLP with statistical feature

The second MLP configuration keeps the same multi-task logic, but changes the input representation. Instead of using a single raw sensor readout, the model receives a vector of statistical descriptors extracted from the temporal evolution of the sensor signals. In this case, each acquisition is transformed into a compact feature vector:

$$x_{stat} = [s_1, s_2, \dots, s_d]$$

where each component represents a statistical descriptor computed from the original signal. This transformation allows the model to work on summarized temporal information rather than on individual CSV rows. The architecture follows the same sequence as the raw-feature MLP. First, the statistical feature vector is provided as input, then the shared hidden layers learn a common representation and finally two output branches predict the gas class and the chemical family class. The main difference with

respect to the raw-feature model is therefore not the learning structure, but the type of information provided to the network.

MLP with Statistical Features

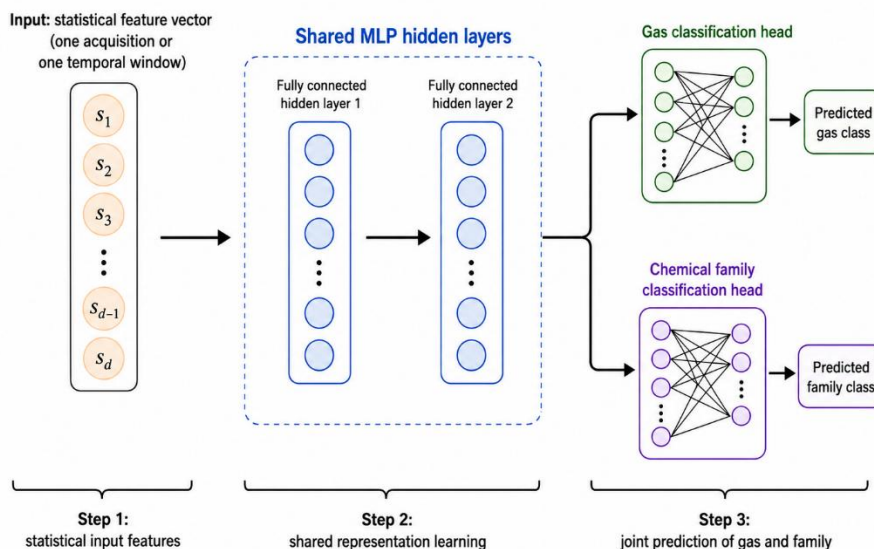


Figure 9: MLP model with statistical features.

4.2.3 RF with raw features

Random Forest was also considered as a supervised classification model. In this configuration each sample is represented by the original sensors' values extracted from CSV files' single rows. Differently from MLP, RF does not implement a neural multi-task architecture with shared layers and multiple output heads, so gas and family classifications are treated as two separate single-task classification problems. The model receives the same input, but two different processes are considered for computing the two labels' predictions.

The functioning of the model is based on an ensemble of decision trees. Each tree learns a sequence of decision rules on the input features, and the final prediction, one for gas and one for family, is obtained by aggregating the outputs of all trees.

Random Forest with Raw Features

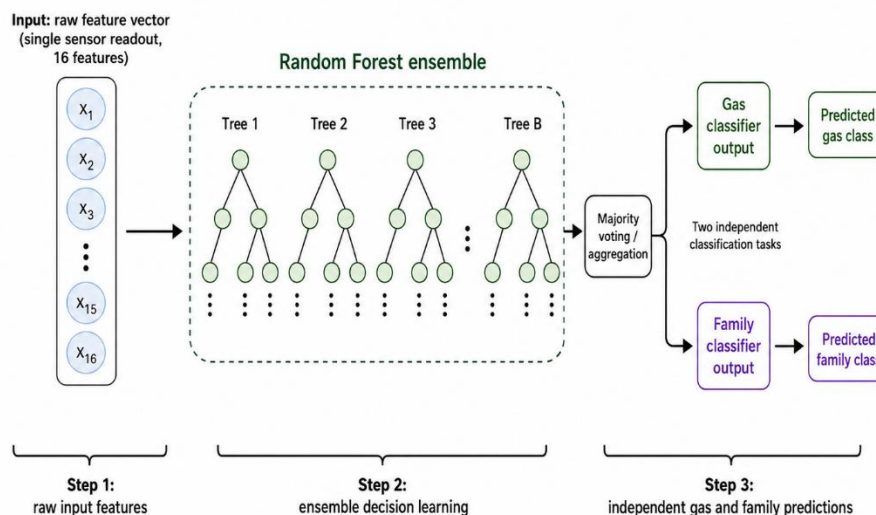


Figure 10: RF model with raw features.

4.2.4 RF with statistical features

In this second configuration of Random Forest classifier, the input is represented by statistical descriptors extracted from the sensor signals, as already seen in the MLP architecture. The model operates on these aggregated features and performs classification through an ensemble of decision trees. As in the previous Random Forest configuration, gas and family prediction are treated as two separate single-task classification tasks. Each forest learns decision rules based on the statistical representation of the data, allowing it to distinguish between gas classes and chemical families. This setup allows the evaluation of how tree-based models behave when provided with summarized temporal information, in particular how the use of this data can influence the effectiveness of RF models, since these descriptors provide more stable and informative patterns than the raw ones, possibly improving the separability of classes through threshold-based decision rules.

Random Forest with Statistical Features

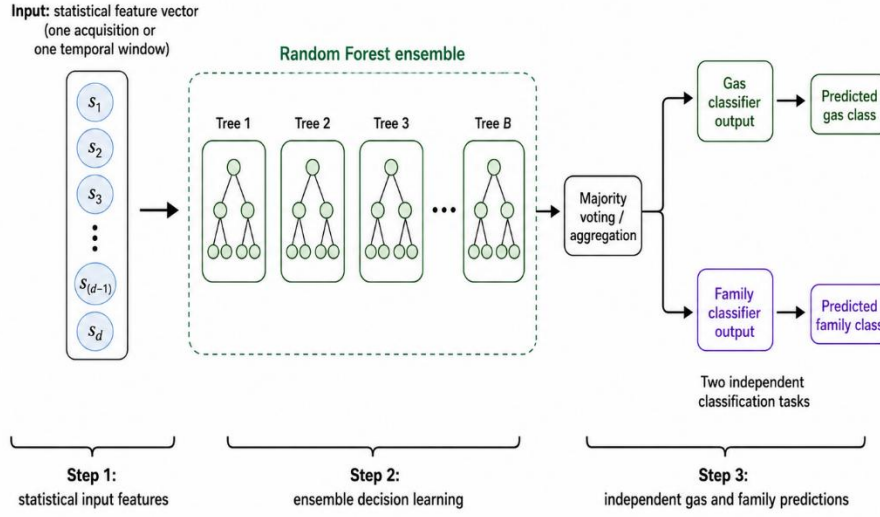


Figure 11: RF model with statistical features.

4.2.5 Window-size model

The window-size model is introduced to analyze the effect of temporal segmentation on the classification process, instead of representing a complete acquisition with a single global descriptor vector, the signal is divided into temporal windows. The general workflow is composed of four steps: first, each CSV acquisition is interpreted as a temporal sequence of sensor readings, second, the sequence is divided into windows of fixed size, third, for each window, statistical descriptors are computed from the sensor signals and finally, the resulting feature vectors are used as input to the classification model. This approach makes it possible to study how much temporal context is needed for gas and family classification. Smaller windows provide more local information and generate more samples from each acquisition, while larger windows summarize longer portions of the signal. This structure enables the comparison between local temporal descriptions and more global representations of the sensor response, and its use with both MLP and Random Forest models makes it possible to evaluate the impact of temporal windowing on their classification task.

Window-Size Model

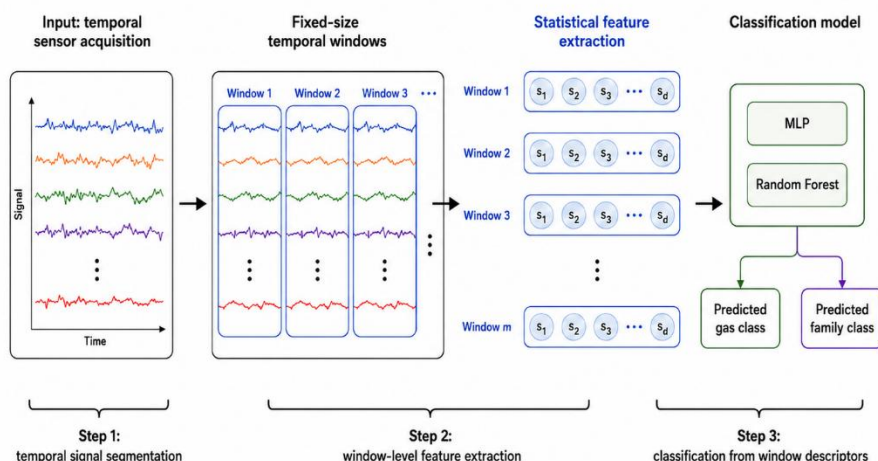


Figure 12: MLP model with windowed features extraction.

4.3 Multi-task cascade architecture

The second group of models is based on cascade strategy. Instead of predicting gas and chemical family in parallel, a sequential dependency has been introduced between the two prediction levels. The main idea is to exploit the chemical family as intermediate information that can support the gas classification process. In the first stage, only the original input data are used to predict the chemical family. Then, in the second stage, the original input features are combined with the family information obtained from the first stage, and this enriched representation is used to perform gas classification. This approach reflects the hierarchical relation between the two labels. Since each gas belongs to a specific chemical family, the family prediction can provide additional contextual information to the gas classifier. The cascade architecture is therefore designed to evaluate whether explicitly using family-level information improves the coherence and effectiveness of gas-level predictions. In the following sections, two cascade configurations are described: the separated cascade and the joint cascade. For each configuration, both hard and soft versions are considered, depending on how the family information is represented and passed to the gas classification stage.

4.3.1 Separated Cascade

The separate cascade is composed of two distinct models trained in two different stages.

The first stage can be represented as:

$$X \rightarrow \text{FamilyMLP} \rightarrow \hat{f}$$

where (X) is the original input feature vector and ($\hat{f}$) is the predicted chemical family. This stage learns a coarse classification task, in which the model assigns each sample to one of the available chemical families.

The second stage receives an enriched input representation obtained by combining the original features with the family information generated by the first model:

$$[X, \hat{f}] \rightarrow \text{GasMLP} \rightarrow \hat{g}$$

where ($\hat{g}$) is the predicted gas class. In this way, the gas classifier does not rely only on the sensor features, but also on an intermediate family-level prediction. In this architecture there is no shared training process between the two stages and no gradient propagation from the gas classifier back to the family one, so that the gas model cannot modify the family model during its own training.

Two variants of the separated cascade are considered, depending on how the predicted family information is represented before being passed to the gas classifier: hard cascade and soft cascade.

Hard Separated Cascade

In the hard separated cascade, the output of the family model is converted into a discrete class prediction. The family logits are first transformed into a predicted class by applying the argmax operation:

$$\hat{f} = \arg \max_c p(f_c | X)$$

The predicted family class is then encoded as a one-hot vector and concatenated with the original input features:

$$z_{hard} = [X, \text{onehot}(\hat{f})]$$

This vector is used as input for the gas classifier:

$$z_{hard} \rightarrow GasMLP \rightarrow \hat{g}$$

The hard cascade therefore provides the gas model with only the final decision of the family classifier, if the family model is uncertain between two or more families, this uncertainty is lost after the argmax operation.

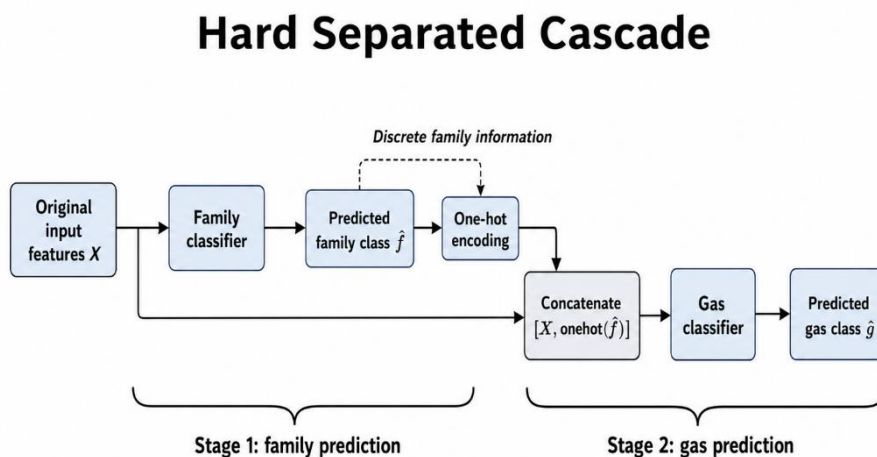


Figure 13: Hard Separated Cascade.

Soft Separated Cascade

In the soft separated cascade, the output of the family model is represented by the full probability distribution over the chemical families. Instead of selecting only the most probable family, the family logits are transformed through a softmax function:

$$p_f = \text{softmax}(\text{FamilyMLP}(X))$$

The resulting probability vector is concatenated with the original input features:

$$z_{soft} = [X, p_f]$$

and used as input for the gas classifier:

$$z_{soft} \rightarrow GasMLP \rightarrow \hat{g}$$

This version provides the gas model with richer information than the hard cascade: the gas classifier receives not only the most likely family, but the confidence distribution over all possible families. Therefore, as in the previous cascade, if the family model is uncertain, this uncertainty is preserved and can be exploited by the second stage.

Soft Separated Cascade

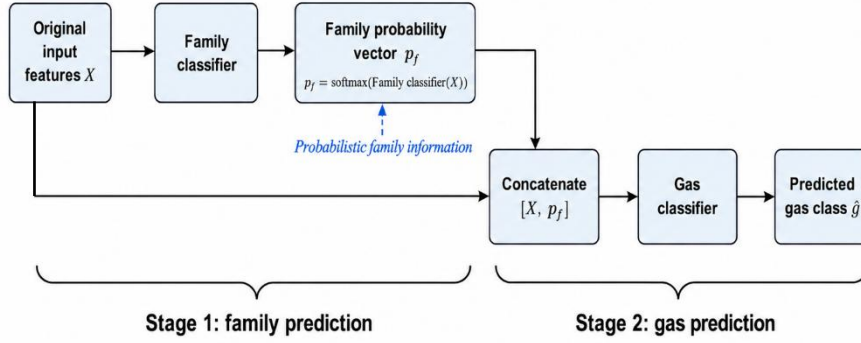


Figure 14: Soft Separated Cascade.

4.3.2 Joint Cascade

The joint cascade architecture keeps the two-stage logic of the cascade approach, but the family and gas prediction modules are trained within a single model. Unlike the separated cascade, where the family classifier is trained first and then used to build the input for the gas classifier, the joint cascade learns both prediction levels simultaneously. The model receives the original input feature vector (X). This input is first processed by the family branch, which produces a family-level prediction. The family information obtained from this branch is then combined with the original input features and passed to the gas branch, which predicts the final gas class.

The general structure can be represented as:

$$X \rightarrow \text{FamilyBranch} \rightarrow \hat{f}$$

$$[X, \hat{f}] \rightarrow \text{GasBranch} \rightarrow \hat{g}$$

where $(\hat{f})$ is the predicted chemical family and $(\hat{g})$ is the predicted gas class. Since both branches belong to the same architecture, the model is optimized jointly. During training, the family output is compared with the true family label, while the gas output is compared with the true gas label. As in the separated cascade, two variants are considered: hard joint cascade and soft joint cascade.

Hard Joint Cascade

In the hard joint cascade, the output of the family branch is converted into a discrete family prediction. The most probable family is selected through an argmax operation:

$$\hat{f} = \arg \max_c p(f_c | X)$$

The predicted family is then transformed into a one-hot vector and concatenated with the original input features:

$$z_{hard} = [X, \text{onehot}(\hat{f})]$$

This enriched vector is used by the gas branch to predict the final gas class. The hard version therefore provides the gas classifier with a compact and discrete representation of the predicted chemical family.

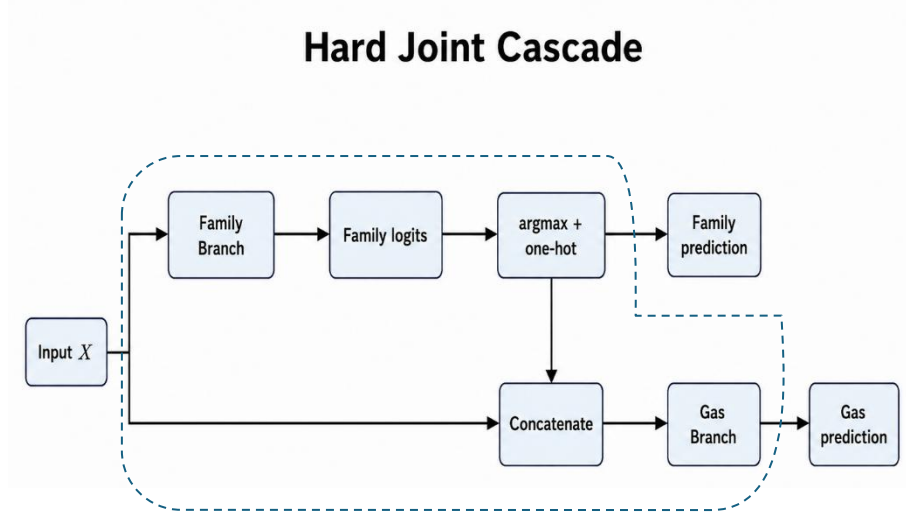


Figure 15: Hard Joint Cascade.

Soft Joint Cascade

In the soft joint cascade, the family information is represented by the complete probability distribution produced by the family branch. Instead of using only the most probable family, the family logits are transformed through a softmax function:

$$p_f = \text{softmax}(\text{FamilyBranch}(X))$$

The probability vector is then concatenated with the original input features:

$$z_{\text{soft}} = [X, p_f]$$

and passed to the gas branch. This representation preserves the uncertainty of the family prediction and provides the gas classifier with more informative input than a single discrete family label.

Soft Joint Cascade

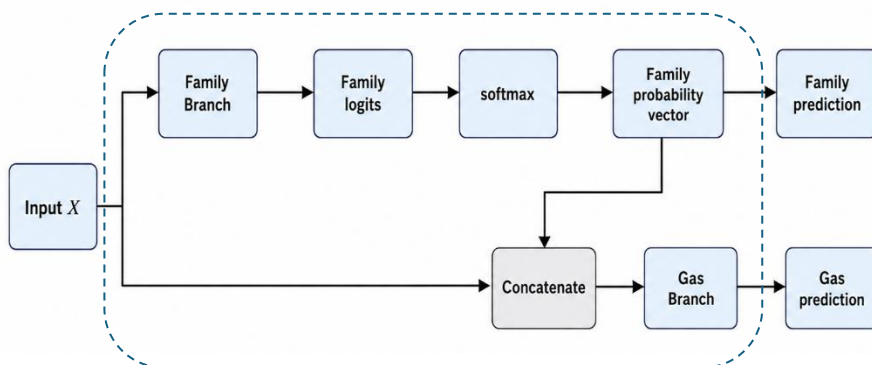


Figure 16: Soft Joint Cascade.

4.4 Multi-task oracle architecture

The oracle architecture is introduced as a controlled model configuration in which the chemical family information is assumed to be already known. Unlike the cascade architectures, the model does not include a family prediction stage, instead the true chemical family associated with each sample is directly provided as additional input to the gas classifier. This architecture is not intended to represent a realistic inference scenario, because in practical applications the true family label would not be available before classification, but its purpose is to evaluate the potential contribution of correct family-level information to the gas prediction task. In this sense, the oracle model provides an upper-bound reference for understanding how much the gas classifier can benefit when the family information is perfectly known.

Hard oracle

The hard oracle model receives two inputs: the original feature vector (X) and the true chemical family encoded as a one-hot vector. The input representation can be written as:

$$z_{oracle} = [X, onehot(f)]$$

where (f) is the true family label associated with the gas sample. This enriched input is passed to an MLP that predicts only the gas class:

$$[X, \text{onehot}(f)] \rightarrow \text{GasMLP} \rightarrow \hat{g}$$

Since the family is given directly as ground truth, the model does not need to learn a family classifier. Consequently, there is no family prediction error and no propagation between a family branch and a gas branch. The only learned task is gas classification conditioned on correct family information. This setting isolates the effect of correct family information and makes it possible to evaluate how much knowledge of chemical family can improve gas classification.

Hard Oracle

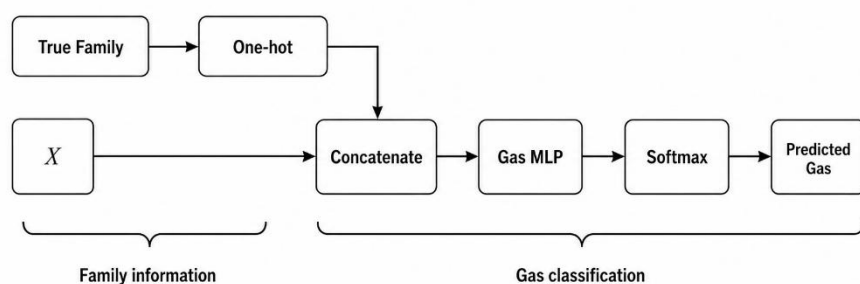


Figure 17: Hard Oracle.

5. Experimental Setup

This chapter describes the experimental setup used to train, configure and evaluate the models presented in the previous chapter. While chapter 4 focused on the architectural structure of each model, this one focuses on the practical aspects of the experimentation, including the software environment, the main libraries, the training pipeline and the parameters used during the tests. At first, the Python environment and the libraries adopted for data processing, model implementation and evaluation are introduced together with the experimental pipeline including data loading, label encoding, features preparation, normalization and temporal split management. Following, main configurations for model families are summarized with loss functions and testing hyperparameters, finally, the command-line options used to customize the experiments and the evaluation metrics adopted to compare the models, namely gas accuracy, family accuracy and gas-family consistency.

5.1 Python Environment and Libraries

The whole experimental pipeline was developed in Python. Python is widely adopted in scientific computing, data science and Machine Learning because it combines a high-level programming interface with a large choice of specialized libraries for numerical computation, data processing and model development [16]. This makes it suitable for implementing the complete workflow of this thesis, from CSV loading and preprocessing to model training, evaluation and result saving. The first group of libraries used in the experiments concerns data handling and numerical computation.

Pandas was used to read the CSV acquisition files through functions such as `read_csv`, and to represent them as `DataFrame` objects [17]. Common operations included column selection, filtering, handling missing values, and splitting the dataset into input features and target labels using indexing operations like `iloc` and `drop`.

NumPy was used for numerical operations and array manipulation, including conversion from DataFrame to numerical arrays (`values` or `to_numpy`), reshaping, concatenation and basic statistical computations [18].

Scikit-learn was used for preprocessing, classical Machine Learning models and evaluation utilities [19]. Label encoding was performed using `LabelEncoder` with methods such as `fit`, `transform` and `inverse_transform`. Feature scaling was handled through `StandardScaler` and `MinMaxScaler`, using `fit` and `transform` to normalize the input space. Scikit-learn was also used to implement Random Forest classifiers through `RandomForestClassifier` [10], relying on methods such as `fit` and `predict`. Evaluation utilities included functions such as `accuracy_score`, `classification_report` and `confusion_matrix` [20], which were used to compute performance metrics and generate evaluation summaries. In the initial phase of the work, the baseline model was entirely implemented using Scikit-learn. However, as the experimentation progressed, all the proposed models were developed using PyTorch, mainly because it allows greater flexibility in defining and customizing loss functions, which was essential for implementing the multi-task and hierarchical learning strategies explored in this thesis.

PyTorch was used for all neural network models [21]. The implemented MLP, cascade and oracle architectures were defined as subclasses of `torch.nn.Module`, using layers such as `nn.Linear`, activation functions like `nn.ReLU`, and dropout layers (`nn.Dropout`). Input data were converted into tensors using `torch.tensor` and organized through custom `Dataset` classes and `DataLoader` objects for batching and shuffling [22]. The training process followed the standard loop based on forward propagation (`model(x)`), loss computation (e.g., `nn.CrossEntropyLoss`), backpropagation (`loss.backward()`) and parameter updates using optimizers such as `torch.optim.Adam` [23] with `optimizer.step()` and `optimizer.zero_grad()`. Additional tensor operations such as `argmax` were used to extract predicted classes from model outputs.

Additional Python modules were used to manage the experimental workflow. The `argparse` module was used to define command-line arguments through `ArgumentParser`, enabling flexible configuration of parameters such as model type, loss

function and hyperparameters [24]. The `json` module was used to load predefined cross-validation splits using `json.load`, while `pathlib` was used for file and directory management through objects such as `Path` [25]. Standard modules such as `random` were used for reproducibility and data shuffling. Finally, `Matplotlib` and `Seaborn` were used to generate graphical outputs, including confusion-matrix heatmaps through functions such as `plt.figure`, `sns.heatmap` and `plt.savefig` [26][27].

5.2 Experimental Pipeline

5.2.1 Data Loading and Label Encoding

Algorithm 1 Data Loading and Label Encoding

Require: Training file list L_{train} , validation file list L_{valid} , dataset root R

Ensure: Feature matrices X_{train} , X_{valid} and encoded labels

- 1: Initialize empty lists for training features and labels
- 2: Initialize empty lists for validation features and labels
- 3: **for** each file in L_{train} **do**
- 4: Read the CSV file from the dataset root R
- 5: Split the table into feature columns X and gas labels y_{gas}
- 6: Keep only numerical feature columns and remove invalid values
- 7: Append X and y_{gas} to the training lists
- 8: **end for**
- 9: **for** each file in L_{valid} **do**
- 10: Read the CSV file from the dataset root R
- 11: Split the table into feature columns X and gas labels y_{gas}
- 12: Keep only numerical feature columns and remove invalid values
- 13: Append X and y_{gas} to the validation lists
- 14: **end for**
- 15: Concatenate all training files into X_{train} and y_{train}^{gas}
- 16: Concatenate all validation files into X_{valid} and y_{valid}^{gas}
- 17: Map gas labels to chemical family labels:

$$y^{family} = map_gas_to_family(y^{gas})$$

- 18: Fit gas encoder on y_{train}^{gas}
 - 19: Fit family encoder on y_{train}^{family}
 - 20: Encode gas labels and family labels for both training and validation sets
 - 21: **return** X_{train} , X_{valid} , y_{train}^{gas} , y_{valid}^{gas} , y_{train}^{family} , y_{valid}^{family}
-

This section describes how the input data are loaded from the CSV files and converted into a format suitable for supervised learning. For each fold, the training and validation file lists are read, and each CSV file is parsed into a table of sensor measurements. The

feature columns are separated from the gas label, keeping only numerical input columns and removing invalid or missing values when necessary.

After loading the gas labels, each gas class is mapped to its corresponding chemical family through the predefined gas-to-family dictionary. Both gas labels and family labels are then encoded into numerical values using label encoders, since the learning algorithms operate on numerical class indices during training.

5.2.2 Feature Preparation and Normalization

Algorithm 2 Feature Preparation and Normalization

Require: Training features X_{train} , validation features X_{valid} , feature mode, scaler type

Ensure: Normalized feature matrices $\tilde{X}_{train}$ and $\tilde{X}_{valid}$

```

1: if statistical feature mode is enabled then
2:   if aggregate-per-file is enabled then
3:     Compute one statistical feature vector for each CSV file
4:   else
5:     Divide each acquisition into temporal windows
6:     Compute statistical descriptors for each window
7:   end if
8:   Replace invalid values such as NaN or infinity with zero
9: end if
10: Align validation columns with training columns

```

$$X_{valid} \leftarrow align_columns(X_{train}, X_{valid})$$

```

11: if scaler type is standard then
12:   Fit a standard scaler on  $X_{train}$ 
13:   Apply the scaler to  $X_{train}$  and  $X_{valid}$ 
14: else if scaler type is min-max then
15:   Fit a min-max scaler on  $X_{train}$ 
16:   Apply the scaler to  $X_{train}$  and  $X_{valid}$ 
17: else
18:   Use the original feature values without scaling
19: end if
20: Convert the resulting matrices to numerical arrays
21: return  $\tilde{X}_{train}, \tilde{X}_{valid}$ 

```

This second part focuses on the construction of the input feature matrix. Depending on the selected configuration, the model can use raw sensor features, rolling-mean features, statistical features extracted from temporal windows, or statistical descriptors computed over the whole CSV file. In the statistical configuration, each acquisition is transformed into a compact numerical representation before being passed to the classifier. After feature extraction, the training and validation matrices are aligned to ensure that they contain the

same columns in the same order. Invalid values such as missing or infinite values are replaced when necessary and finally, normalization is applied using either standard scaling, min-max scaling, or no scaling, depending on the selected command-line option. The scaler is fitted only on the training data and then applied to the validation data, avoiding information leakage.

5.2.3 Train/Validation Split Management

Algorithm 3 Train/Validation Split Management

Require: Split file S , dataset root R , optional external test root R_{test}

Ensure: Training and validation file lists for each fold

1: **if** external test root R_{test} is provided **then**

2: Build the training file list from R

3: Build the test file list from R_{test}

4: Create a single split named *external_test*

5: **else**

6: Load the predefined split file:

$$splits \leftarrow json.load(S)$$

7: **end if**

8: **for** each fold in *splits* **do**

9: Extract the training file list:

$$L_{train} \leftarrow fold[train]$$

10: Extract the validation file list:

$$L_{valid} \leftarrow fold[valid]$$

11: Load training data using L_{train}

12: Load validation data using L_{valid}

13: Apply preprocessing, training and evaluation for the current fold

14: **end for**

This third section describes how the experimental folds are managed. The experiments mainly rely on predefined train/validation splits stored in a JSON file. For each fold, the script extracts the list of training files and the list of validation files, ensuring that all models are evaluated using the same partitions. The same pipeline also supports an external test configuration, in this case, the CSVs are loaded from the training data root, while the test files are loaded from a separate test directory. This makes it possible to use either the predefined cross-validation protocol or a separate external test set without changing the general structure of the experimental workflow.

5.2.4 Model Training and Prediction Flow

Algorithm 4 Model Training and Prediction Flow

Require: Training data $\tilde{X}_{train}$, validation data $\tilde{X}_{valid}$, encoded labels, model configuration

Ensure: Gas predictions, family predictions and evaluation metrics

- 1: Create training dataset and validation dataset
- 2: Create mini-batch data loaders
- 3: Initialize the model
- 4: Initialize the optimizer
- 5: Select the loss function according to the chosen loss mode
- 6: **for** each epoch $e = 1, \dots, E$ **do**
- 7: **for** each mini-batch (X, y_{gas}, y_{family}) **do**
- 8: Reset optimizer gradients
- 9: Compute model outputs:

$$\hat{y}_{gas}, \hat{y}_{family} = model(X)$$

- 10: Compute gas classification loss
- 11: Compute family classification loss
- 12: Combine the two losses:

$$\mathcal{L} = \mathcal{L}_{gas} + \alpha \mathcal{L}_{family}$$

- 13: **if** hierarchical loss is enabled **then**
 - 14: Compute hierarchical consistency penalty
 - 15: Add the penalty to the total loss
 - 16: **end if**
 - 17: Backpropagate the loss
 - 18: Update model parameters
 - 19: **end for**
 - 20: **end for**
 - 21: Set the model to evaluation mode
 - 22: **for** each validation mini-batch **do**
 - 23: Compute gas and family logits
 - 24: Convert logits into predicted classes using argmax
 - 25: Store predictions
 - 26: **end for**
 - 27: Decode predicted labels
 - 28: Compute gas accuracy, family accuracy and gas-family consistency
 - 29: **return** evaluation metrics
-

This last section summarizes the common training and prediction procedure used by the neural models. After preprocessing, the feature matrices and encoded labels are converted into dataset objects and mini-batch data loaders, then the selected model is initialized together with the optimizer and the loss function. During training, each mini-batch is passed through the system to obtain gas and, when required, family predictions. The loss is computed from the model outputs and the true labels, then backpropagation is used to update the parameters. At the end of training, the model is evaluated on the validation set. The output logits are converted into predicted classes using the argmax operation, and the predictions are compared with the true labels to compute the final evaluation metrics.

5.3 Model Configuration

5.3.1 MLP-Based Models

Algorithm 5 MLP-Based Multi-Task Model Configuration

Require: Input dimension d , number of gas classes C_g , number of family classes C_f

Ensure: Multi-task MLP model

1: Define shared backbone:

$$d \rightarrow 128 \rightarrow 64$$

2: Define gas output head:

$$64 \rightarrow C_g$$

3: Define family output head:

$$64 \rightarrow C_f$$

4: **for** each mini-batch X **do**

5: Compute shared representation:

$$z = \text{SharedMLP}(X)$$

6: Compute gas logits:

$$\hat{y}_{gas} = \text{GasHead}(z)$$

7: Compute family logits:

$$\hat{y}_{family} = \text{FamilyHead}(z)$$

8: **end for**

9: **return** gas logits and family logits

The standard multi-task architecture receives the input feature vector and processes it through a shared backbone composed of two fully connected layers. The shared representation is then passed to two separate output heads: one for gas classification and one for chemical family classification. The shared backbone uses two hidden layers with 128 and 64 neurons, respectively. The gas output head produces the logits associated with the gas classes, while the family output head produces the logits associated with the chemical family classes. The model is trained using mini-batches and an optimizer, with a loss function combining gas and family classification errors.

5.3.2 Random Forest Models

Algorithm 6 Random Forest Model Configuration

Require: Training features $\tilde{X}_{train}$, validation features $\tilde{X}_{valid}$, gas labels y_{gas} , family labels y_{family}

Ensure: Gas predictions and family predictions

- 1: Initialize gas Random Forest classifier
- 2: Initialize family Random Forest classifier
- 3: Train gas classifier:

$$RF_{gas}.fit(\tilde{X}_{train}, y_{gas})$$

- 4: Train family classifier:

$$RF_{family}.fit(\tilde{X}_{train}, y_{family})$$

- 5: Predict gas labels:

$$\hat{y}_{gas} = RF_{gas}.predict(\tilde{X}_{valid})$$

- 6: Predict family labels:

$$\hat{y}_{family} = RF_{family}.predict(\tilde{X}_{valid})$$

- 7: **return** $\hat{y}_{gas}, \hat{y}_{family}$
-

Two independent Random Forest classifiers were trained: one using the encoded gas labels and one using the encoded family labels. Unlike the MLP-based models, Random Forest does not use shared hidden representations or backpropagation. The models are trained using the scaled input features and then applied to the validation set to obtain gas and family predictions.

5.3.3 Cascade and Oracle Models

Algorithm 7 Cascade and Oracle Model Configuration

Require: Input features X , family mode, cascade type

Ensure: Gas prediction $\hat{y}_{gas}$ and, when available, family prediction $\hat{y}_{family}$

1: **if** separated cascade **then**

2: Train family model:

$$X \rightarrow FamilyMLP \rightarrow \hat{y}_{family}$$

3: Convert family output into hard or soft family features

4: Train gas model:

$$[X, family_features] \rightarrow GasMLP \rightarrow \hat{y}_{gas}$$

5: **else if** joint cascade **then**

6: Compute family logits:

$$family_logits = FamilyBranch(X)$$

7: Convert family logits into hard or soft family features

8: Concatenate original input and family features:

$$z = [X, family_features]$$

9: Predict gas class:

$$z \rightarrow GasBranch \rightarrow \hat{y}_{gas}$$

10: **else if** hard oracle **then**

11: Convert true family label into one-hot vector:

$$f_{true} = onehot(y_{family})$$

12: Concatenate original input and true family information:

$$z = [X, f_{true}]$$

13: Predict gas class:

$$z \rightarrow GasMLP \rightarrow \hat{y}_{gas}$$

14: **end if**

15: **return** model predictions

Cascade models were implemented to explicitly use family-level information during gas classification. In the separated cascade, two different models are trained. The first model predicts the chemical family from the input features. The second model receives the original input features concatenated with the family information produced by the first model and predicts the gas class.

In the joint cascade, the same logic is implemented inside a single neural architecture. The family branch first produces family logits, which are converted into hard or soft family features. These features are concatenated with the original input and passed to the gas branch.

The hard oracle model follows a similar input structure, but it does not predict the family. Instead, the true family label is converted into a one-hot vector and concatenated with the input features. The resulting vector is used only for gas classification.

5.3.4 Window-Based Models

Algorithm 8 Window-Based Model Configuration

Require: CSV acquisition A , window size w , step s , aggregation mode

Ensure: Statistical feature vectors

- 1: **if** aggregate-per-file mode is enabled **then**
- 2: Compute one statistical feature vector from the whole acquisition:

$$x_{stat} = \text{Aggregate}(A)$$

- 3: **else**
- 4: Initialize empty list of feature vectors
- 5: **for** each temporal window $A_{i:i+w}$ with step s **do**
- 6: Compute statistical descriptors:

$$x_i = \text{Aggregate}(A_{i:i+w})$$

- 7: Append x_i to the feature list
 - 8: **end for**
 - 9: **end if**
 - 10: Use the resulting statistical feature vectors as model input
 - 11: **if** model is MLP **then**
 - 12: Train the multi-task MLP on statistical features
 - 13: **else if** model is Random Forest **then**
 - 14: Train independent gas and family Random Forest classifiers
 - 15: **end if**
 - 16: **return** model predictions
-

In window-based models, instead of using each raw row independently or aggregating the whole file into a single vector, each CSV acquisition can be divided into temporal windows. Statistical descriptors are then computed for each window and used as model inputs. The same feature extraction procedure also supports an aggregate-per-file configuration, where a single statistical vector is computed for the whole CSV file, after feature extraction, the resulting vectors are passed to either the MLP model or the Random Forest classifiers.

5.5 Hyperparameters Setting

The experimental setup exposes the main training and model parameters. This makes it possible to run experiments with different configurations, such as standard, focal and hierarchical loss, hard or soft cascade inputs, raw or statistical features, and MLP or Random Forest classifiers.

Table 1: Main hyperparameters used for MLP-based models.

Hyperparameter	Value
Epochs	Configurable
Batch size	64
Learning rate	0.001
Optimizer	Adam
Hidden layers	128, 64
Activation function	ReLU
Family loss weight α	Configurable
Random seed	42
Scaler	Standard

For the neural models, the main training parameters included the number of epochs, batch size, learning rate, optimizer and the weight used to balance the different loss components. These parameters were selected to ensure stable training and consistent comparisons across the various model configurations. In general, the same set of values was adopted across the MLP-based models and the cascade configurations to maintain comparability between experiments.

Table 3: Random Forest hyperparameters.

Hyperparameter	Value
Number of estimators	300
Maximum depth	None
Minimum samples per leaf	1
Class balancing	Optional

For the Random Forest experiments, the statistical-feature script allowed the classifier backend to be selected through the `--model` argument. When `rf` was selected, two Random Forest classifiers were trained, one for gas classification and one for family classification. The main Random Forest hyperparameters exposed by the script were the number of estimators, maximum depth, minimum number of samples per leaf and the optional class-balancing flag.

Table 4: Window-based feature extraction parameters.

Hyperparameter	Value
Statistical window size	Configurable
Window step	Configurable
Number of internal windows	Configurable
Aggregate per file	Enabled / Disabled

For the window-based experiments, the most relevant parameters were the statistical window size, the step between consecutive windows and the number of internal sub-windows used during statistical aggregation. The script also allowed an aggregate-per-file configuration, in which a single statistical feature vector was computed for each CSV file instead of extracting multiple temporal windows.

Table 5: Cascade-specific hyperparameters.

Hyperparameter	Value
Cascade input	Hard / Soft
Hard representation	One-hot family vector
Soft representation	Family probability vector
OOF folds	Configurable

Finally, cascade models introduced the additional cascade-input parameter, which determines how family information is passed to the gas classifier. In the hard configuration, the predicted family is converted into a one-hot vector. In the soft configuration, the full probability distribution over family classes is used. The separated cascade script also includes an out-of-fold option, controlled by the number of inner folds.

5.6 Loss Function Settings

Algorithm 9 Loss Function Selection

Require: Gas logits, family logits, gas labels, family labels, loss mode

Ensure: Total training loss $\mathcal{L}$

1: **if** loss mode is standard **then**

2: Compute gas cross-entropy loss:

$$\mathcal{L}_{gas}$$

3: Compute family cross-entropy loss:

$$\mathcal{L}_{family}$$

4: Combine the two losses:

$$\mathcal{L} = \mathcal{L}_{gas} + \alpha \mathcal{L}_{family}$$

5: **else if** loss mode is focal **then**

6: Compute focal loss for gas prediction

7: Compute focal loss for family prediction

8: Combine the two losses:

$$\mathcal{L} = \mathcal{L}_{gas}^{focal} + \alpha \mathcal{L}_{family}^{focal}$$

9: **else if** loss mode is hierarchical **then**

10: Compute gas and family classification losses

11: Compute the hierarchical consistency penalty:

$$\mathcal{L}_{hier}$$

12: Add the penalty to the total loss:

$$\mathcal{L} = \mathcal{L}_{gas} + \alpha \mathcal{L}_{family} + \lambda_{hier} \mathcal{L}_{hier}$$

13: **end if**

14: **return** $\mathcal{L}$

The neural models were trained using different loss-function configurations. The choice of the loss function was controlled through the `--loss-mode` command-line argument, which allowed three possible modes: standard, focal and hierarchical. These configurations were used to evaluate how different optimization objectives affect gas classification, family classification and gas-family consistency.

The standard configuration uses cross-entropy loss. In the multi-task models, two losses are computed: one for gas classification and one for family classification. The total loss is obtained by combining them through the family-loss weight (α):

$$L = L_{gas} + \alpha L_{family}$$

The focal-loss configuration replaces the standard cross-entropy criterion with focal loss. This loss reduces the contribution of easy samples and gives more importance to difficult

or misclassified samples. Starting from the cross-entropy loss (L_{CE}), the focal loss is computed as:

$$(1 - p_t)^\gamma L_{CE}$$

where ($p_t = e^{-L_{CE}}$) represents the confidence associated with the correct class, and (γ) controls the strength of the focusing effect.

The hierarchical configuration adds a gas-family consistency penalty to the classification loss. The objective is to penalize predictions in which the gas output and the family output are not chemically coherent, in fact, if the predicted gas and predicted family are inconsistent, an additional penalty is added to the loss. The hierarchical loss used in the experiments can be summarized as:

$$L_{tot} = L_{gas} + \alpha L_{family} + \lambda_{hier} L_{hier}$$

where (λ_{hier}) controls the contribution of the hierarchical penalty. The penalty is based on three cases [28]:

$$L_{hier}^{(i)} = \begin{cases} 0, & \text{if the gas prediction is correct} \\ \alpha_i, & \text{if the gas is wrong but the family is coherent} \\ \gamma_{hier} \alpha_i, & \text{if gas and family are incoherent} \end{cases}$$

with:

$$\alpha_i = -\beta \log(p_{\hat{g}_i})$$

where ($p_{\hat{g}_i}$) is the predicted probability of the selected gas class, (β) scales the penalty according to prediction confidence, and (γ_{hier}) increases the penalty when the predicted gas and predicted family are inconsistent.

Table 2: Loss-function hyperparameters.

Hyperparameter	Value
Loss modes	Standard / Focal / Hierarchical
Focal loss γ	Configurable
Hierarchical loss λ_{hier}	Configurable
Hierarchical loss β	Configurable
Hierarchical loss γ_{hier}	Configurable

5.7 Command-Line Arguments

The experimental scripts were designed to be executed from the command line, allowing the main aspects of each experiment to be configured without modifying the source code. This approach made it possible to reuse the same scripts with different datasets, feature representations, model types, loss functions and output options.

Table 6: Main command-line arguments used in the experimental scripts.

Argument	Values / Default	Description
Dataset and split arguments		
<code>--splits</code>	JSON file / None	Path to the JSON file containing the predefined train/validation folds.
<code>--data-root</code>	Path / .	Root directory containing the training CSV files.
<code>--test-root</code>	Path / None	Optional external test directory. If provided, training data are loaded from <code>--data-root</code> and test data from <code>--test-root</code> .
<code>--path-template</code>	{class_name}/{file}	Template used to build the relative path of each CSV file from the dataset root.
<code>--only-fold</code>	Fold name / None	Runs only one selected fold from the split file. Used in the statistical-feature script.
<code>--max-files-per-class</code>	Integer / None	Optional limit on the number of files loaded for each class. Mainly useful for debugging or reduced experiments.
<code>--drop-unseen-test-labels</code>	Flag	Drops test samples whose gas label is not present in the training set, enabling closed-set evaluation on external test data.
CSV loading arguments		
<code>--sep</code>	Separator / None	CSV column separator. For the dataset used in this thesis, the separator is typically <code>;</code> .
<code>--decimal</code>	Character / .	Decimal separator used when reading numerical values from CSV files.
<code>--index-col</code>	Column / None	Optional index column used when loading CSV files.
<code>--label-col</code>	Column name / None	Optional name of the column containing the label. If not specified, the last column is used as label.
Feature preparation arguments		
<code>--rolling-window</code>	Integer / 1	Number of consecutive rows averaged before training and testing. A value of 1 disables rolling averaging.
<code>--rolling-step</code>	Integer / 1	Step between consecutive rolling windows.
<code>--scaler</code>	standard, minmax, none	Feature scaling strategy. Standard scaling applies z-score normalization, min-max scaling maps features to <code>[0, 1]</code> , and none disables scaling.
<code>--use-stats-features</code>	Flag	Enables statistical feature extraction instead of using raw or rolling numerical rows. Used in cascade and joint-cascade scripts that support statistical features.

Argument	Values / Default	Description
--stats-window-size	Integer / 20	Number of consecutive rows used to compute one statistical feature vector.
--stats-step	Integer / 20	Step between consecutive statistical windows.
--stats-n-windows	Integer / 4	Number of internal sub-windows used during statistical aggregation.
--aggregate-per-file	Flag	Computes one statistical feature vector for each complete CSV file instead of extracting multiple temporal windows.
--verbose-files	Flag	Prints the files processed during statistical feature extraction.
Neural-network training arguments		
--epochs	Integer / 400	Maximum number of training epochs. In the main experiments, this value was often set to 100.
--batch-size	Integer / 64	Mini-batch size used by the PyTorch DataLoader.
--lr	Float / 0.001	Learning rate used by the optimizer.
--alpha-family	Float / 1.0	Weight assigned to the family classification loss in multi-task models. In separated cascade models, it is kept for compatibility but not used as a true joint loss weight.
--tol	Float / 1e-4	Minimum improvement threshold used for early stopping based on training loss.
--n-iter-no-change	Integer / 10	Number of consecutive epochs without sufficient improvement before stopping training.
--seed	Integer / 42	Random seed used to make experiments reproducible.
Loss-function arguments		
--loss-mode	standard, hierarchical	focal, Selects the loss-function configuration used during training.
--focal-gamma	Float / 2.0	Focusing parameter of the focal loss.
--lambda-hier	Float / 1.0	Weight of the hierarchical consistency penalty.
--hier-beta	Float / 1.0	Scaling factor used inside the hierarchical penalty.
--hier-gamma	Float / 2.0	Multiplicative factor used to increase the penalty for gas-family inconsistency.
Random Forest arguments		
--model	mlp, rf	Selects the classifier backend in the statistical-feature script. mlp uses the PyTorch multi-task MLP, while rf trains two Random Forest classifiers.
--rf-estimators	Integer / 300	Number of trees used in the Random Forest classifier.
--rf-max-depth	Integer / None	Maximum depth of each tree. If None, trees are expanded until the stopping criteria are met.
--rf-min-samples-leaf	Integer / 1	Minimum number of samples required in a leaf node.
--rf-balanced	Flag	Enables class balancing for the Random Forest classifiers.
Cascade-specific arguments		
--cascade-input	hard, soft	Controls how family information is passed to the gas classifier. The hard mode uses a one-hot family vector, while the soft mode uses family probabilities.
--oof-folds	Integer / 5	Number of inner folds used to generate out-of-fold family predictions in the separated cascade.
--use-oof	Flag	Enables out-of-fold family predictions for gas-model training in the separated cascade.
--no-oof	Flag	Compatibility flag used to explicitly disable out-of-fold family predictions.
Output and saving arguments		
--save-model	Path / None	Optional path used to save the trained model.
--save-report	Path / None	Optional path used to save the CSV report containing fold-level results.
--save-heatmap	Flag	Enables saving of confusion-matrix heatmaps.
--heatmap-dir	Path / .	Directory where the heatmap images are saved.

5.8 Evaluation Metrics

The models were evaluated using three main metrics: gas accuracy, family accuracy and gas-family consistency. These metrics were computed independently for each fold and then aggregated across all folds using mean and standard deviation. This evaluation strategy made it possible to compare the models both in terms of fine-grained gas classification and higher-level chemical coherence.

5.8.1 Gas Accuracy

Gas accuracy measures the percentage of samples for which the predicted gas label is equal to the true gas label. Given (N) validation samples, gas accuracy is defined as:

$$\frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{g}_i = g_i)$$

where (g_i) is the true gas label, ($\hat{g}_i$) is the predicted gas label, and ($\mathbb{I}(\cdot)$) is an indicator function equal to 1 when the condition is true and 0 otherwise.

5.8.2 Family Accuracy

Family accuracy measures the percentage of samples for which the predicted chemical family is equal to the true chemical family. It is computed as:

$$\frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{f}_i = f_i)$$

where (f_i) is the true family label and ($\hat{f}_i$) is the predicted family label.

This metric evaluates the model at a coarser classification level. It is useful because some gases may be difficult to distinguish individually, but their chemical family can still be predicted correctly. In the hard oracle configuration, the family label is provided as true input information; therefore, the family accuracy is equal to 1 by definition.

5.8.3 Gas-Family Consistency

Gas-family consistency evaluates whether the predicted gas and the predicted family are chemically coherent. Since each gas belongs to a specific family, the predicted gas can be mapped to its expected family. The prediction is considered consistent when this expected family matches the predicted family.

Formally, let $(F(\hat{g}_i))$ be the family associated with the predicted gas $(\hat{g}_i)$, so the consistency score is defined as:

$$\frac{1}{N} \sum_{i=1}^N \mathbb{I}(F(\hat{g}_i) = \hat{f}_i)$$

This value measures whether the gas and family predictions are compatible with the predefined gas-to-family mapping. Therefore, a model can have a wrong gas prediction but still produce a consistent output if the predicted gas belongs to the predicted family. In the hard oracle configuration, the consistency is computed by comparing the family associated with the predicted gas against the true family label, since the family is not predicted by a separate output branch.

5.8.4 Mean and Standard Deviation Across Folds

Since the experiments were performed using a cross-validation protocol, each metric was first computed separately for every fold, then the final performance was summarized using mean and standard deviation across the (K) folds.

For a generic metric (m), the mean score is computed as:

$$\frac{1}{K} \sum_{k=1}^K m_k$$

where (m_k) is the metric value obtained on fold (k). The standard deviation is computed as:

$$\sqrt{\frac{1}{K-1} \sum_{k=1}^K (m_k - \bar{m})^2}$$

The mean provides the average performance of the model across the cross-validation folds, while the standard deviation indicates how stable the model is across different train/validation partitions. Lower standard deviation values indicate more consistent behavior across folds.

6. Experimental Results

This chapter presents and discusses the experimental results obtained from the models described in Chapter 4 and configured according to the setup defined in Chapter 5. The goal is to compare the different classification strategies in terms of gas accuracy, family accuracy and gas-family consistency.

6.1 Industrial Baseline Model

The industrial baseline model represents the reference point for all the following experiments. It obtains a mean gas accuracy of 58.27% across the 10 cross-validation folds. The result shows that the baseline MLP is able to learn discriminative information from the 16 raw sensor features, but its performance remains limited when each sensor row is treated as an independent sample. Since the baseline model predicts only the gas class, family accuracy and gas-family consistency are not directly evaluated.

Table 7: Accuracy values obtained by the industrial baseline model across the 10 cross-validation folds.

Fold	Accuracy
fold_1	0.5373
fold_2	0.5787
fold_3	0.5726
fold_4	0.5488
fold_5	0.6031
fold_6	0.6779
fold_7	0.5434
fold_8	0.5855
fold_9	0.5809
fold_10	0.5987
Mean	0.5827
Std	0.0403

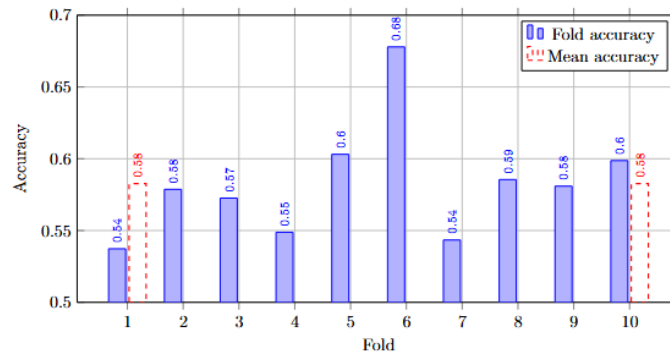


Figure 18: Accuracy values obtained by the industrial baseline model across the 10 cross-validation folds. The dashed red line indicates the mean accuracy.

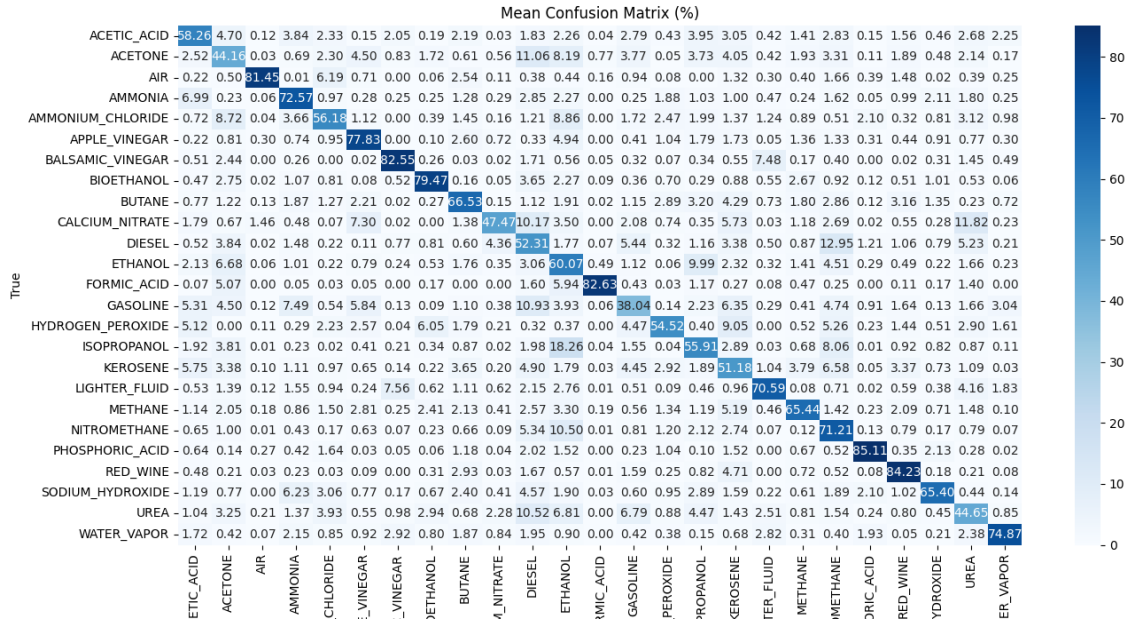
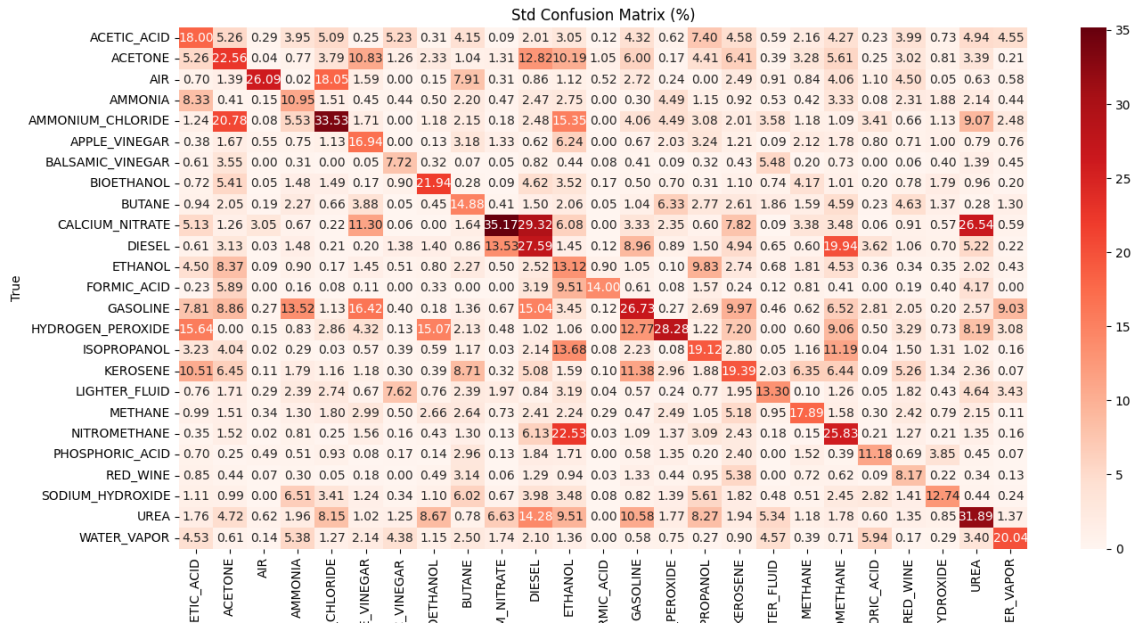


Figure 19: Confusion matrix heatmap of mean gas accuracy obtained by the 10 cross validation folds – MLP classifier.



Industrial Baseline Model with Random Forest Classifier

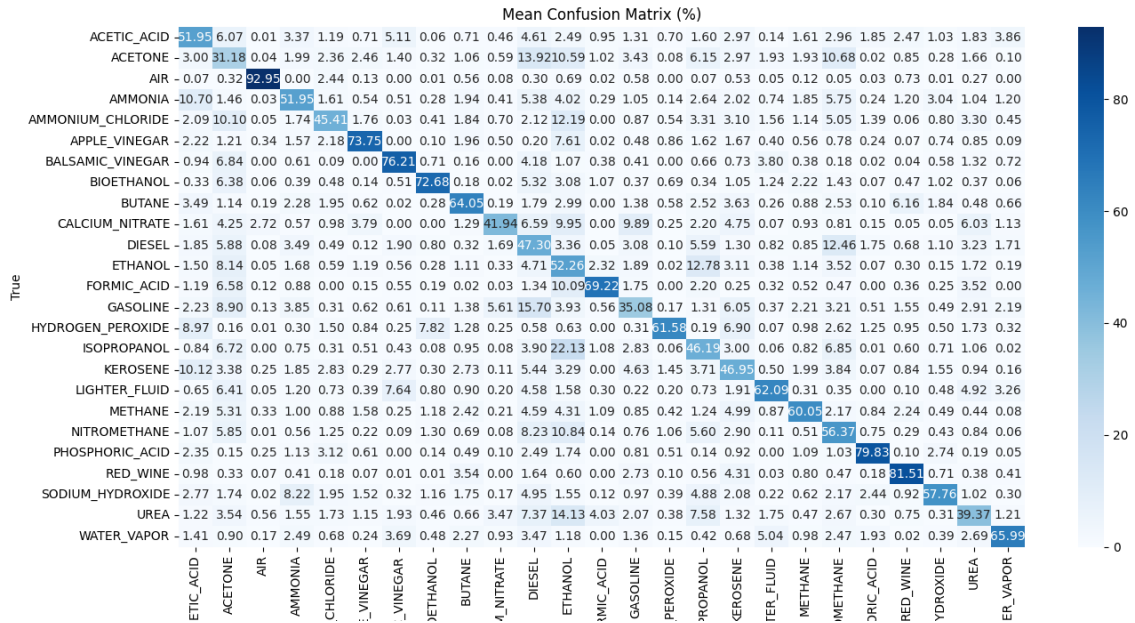
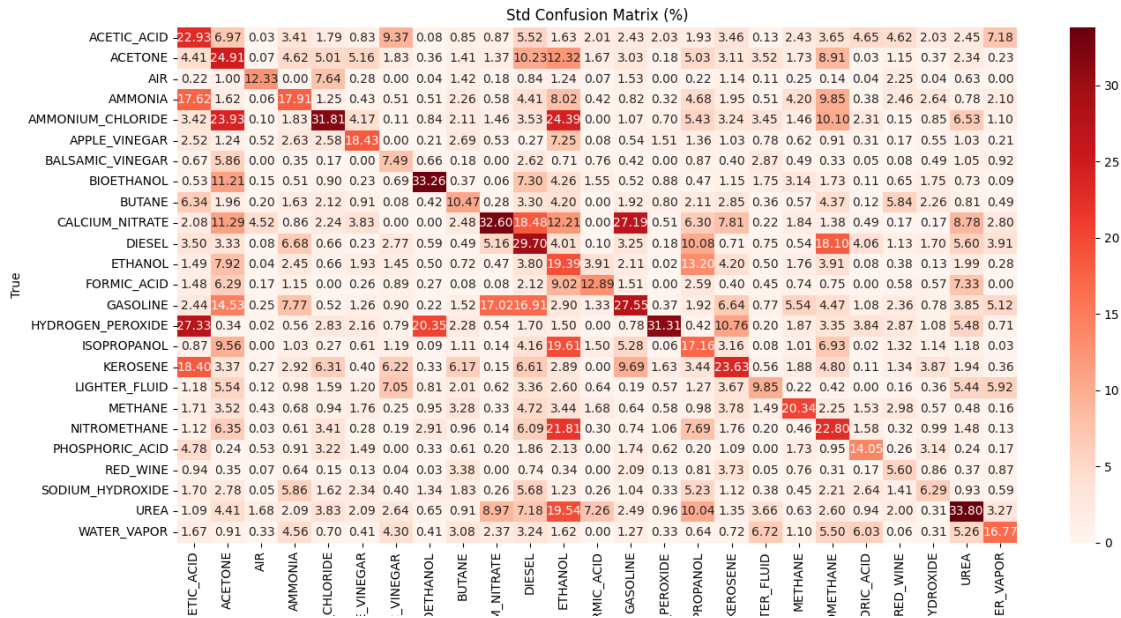


Figure 21: Confusion matrix heatmap of mean gas accuracy obtained by the 10 cross validation folds – RF classifier.



configuration, the tree-based model does not benefit from the temporal structure of the signal and operates only on instantaneous sensor values.

6.2 Experimental Models' Results

MLP – Standard, Hierarchical and Focal Loss Functions Raw Features

Table 10: MLP loss-function configurations and parameters.

Loss configuration	Gas acc. (%)	Family acc. (%)	Parameters
Focal 1	57.96	63.69	$\alpha = 1, \gamma = 2$
Focal 2	58.96	63.54	$\alpha = 0.5, \gamma = 1.5$
Focal 3	57.05	62.51	$\alpha = 1, \gamma = 0.5$
Hierarchical 1	59.17	64.19	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$
Hierarchical 2	58.45	64.32	$\alpha = 1, \beta = 0.5, \gamma_{hier} = 3, \lambda_{hier} = 0.7$
Hierarchical 3	59.51	65.02	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 2, \lambda_{hier} = 0.3$
Standard 1	59.44	64.84	$\alpha = 1$
Standard 2	59.83	64.83	$\alpha = 0.5$
Standard 3	59.08	64.12	$\alpha = 2$

The focal loss configuration produces results comparable to the other loss functions, but does not lead to a clear improvement in either gas or family accuracy in this setting.

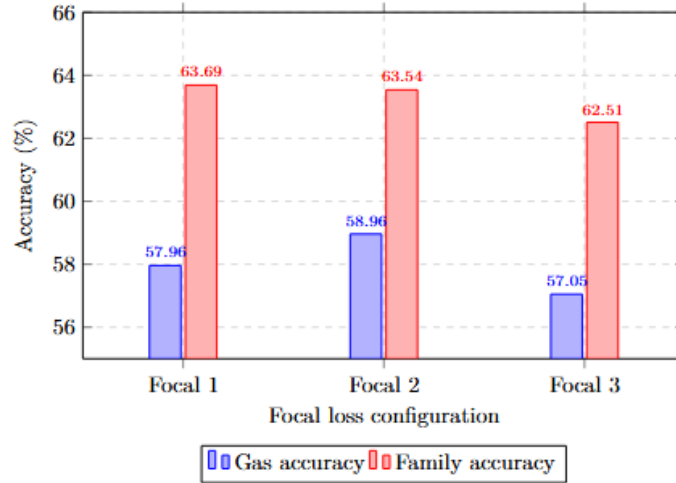


Figure 23: MLP performance with focal loss configuration.

The hierarchical loss configuration provides the best family accuracy, reaching 65.02%, indicating that explicitly modelling the relationship between gas and family helps improve family-level predictions.

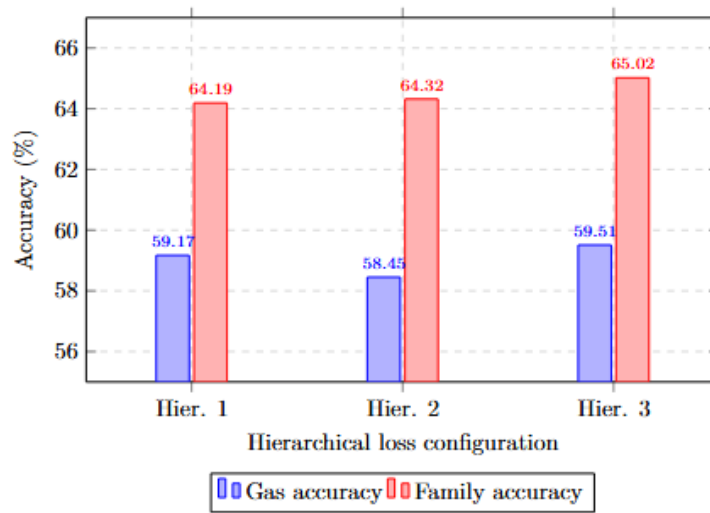


Figure 24: MLP performance with hierarchical loss configuration.

The standard loss configuration achieves the highest gas accuracy, reaching 59.83%, showing that it remains the most effective choice for optimizing direct gas classification on raw features.

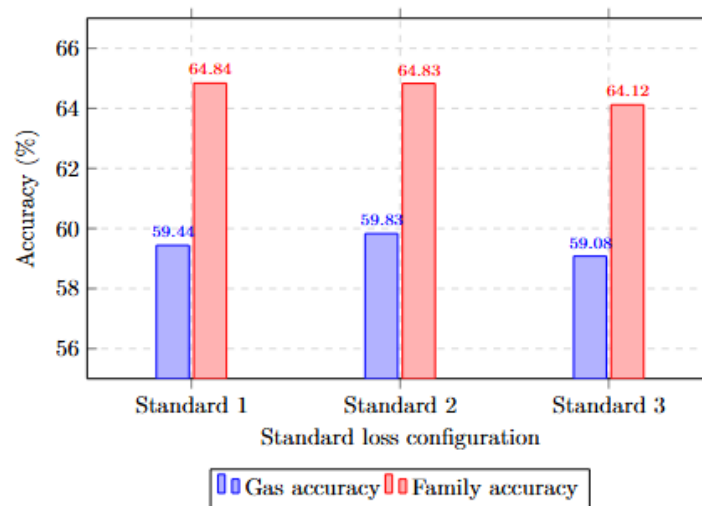


Figure 25: MLP performance with standard loss configuration.

The comparison between standard, hierarchical and focal loss on raw features shows that changing the loss function alone does not produce a substantial improvement in classification performance. The best gas accuracy is obtained with the standard loss configuration, reaching 59.83%, while the best family accuracy is obtained with the hierarchical configuration, reaching 65.02%.

Overall, the three loss families produce similar results. This indicates that the main limitation is not only related to the optimization objective, but also to the information contained in the input representation.

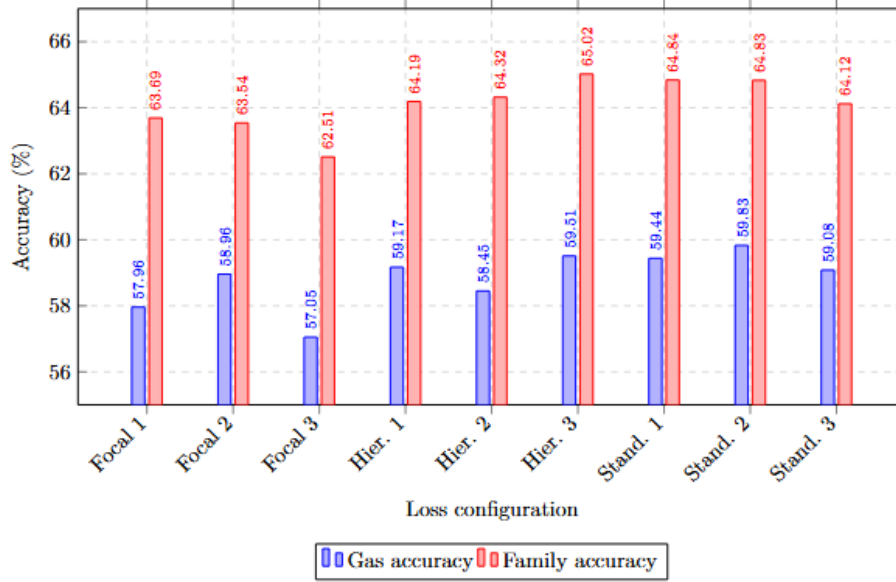


Figure 26: MLP performance for all loss-function configurations.

Table 9: Average MLP performance grouped by loss type.

Loss type	Average gas accuracy (%)	Average family accuracy (%)
Focal	57.99	63.25
Hierarchical	59.04	64.51
Standard	59.45	64.60

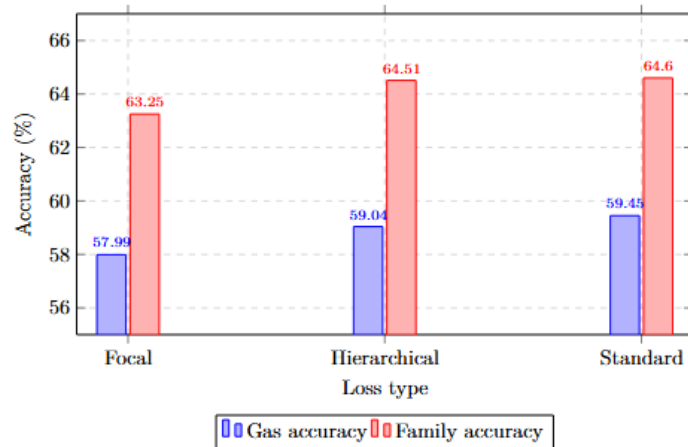


Figure 27: MLP mean performance for all loss-function types.

MLP – Standard, Hierarchical and Focal Loss Functions - Raw Features – Rolling Mean Input

The rolling-mean experiments show that smoothing the raw signal does not clearly improve the performance of the MLP. The best result is obtained with the standard loss and a window size of 5, reaching 59.52% gas accuracy and 63.93% family accuracy. These values are close to the raw multi-task MLP results and do not represent a significant improvement over the baseline. Moreover, increasing the rolling-window size generally reduces accuracy. This suggests that large smoothing windows may remove local variations of the sensor response that are useful for distinguishing between gases. Therefore, rolling mean preprocessing provides a more stable signal, but it does not appear to be the most informative representation for this classification task.

Table 11: MLP results with different loss functions and rolling-window sizes.

Loss configuration	Window size	Gas acc. (%)	Family acc. (%)	Parameters
Focal 5	5	58.12	63.57	$\alpha = 1, \gamma = 2$
Focal 10	10	57.50	62.04	$\alpha = 1, \gamma = 2$
Focal 20	20	54.69	59.74	$\alpha = 1, \gamma = 2$
Focal 50	50	51.60	57.95	$\alpha = 1, \gamma = 2$
Hierarchical 5	5	57.60	63.20	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$
Hierarchical 10	10	56.96	61.74	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$
Hierarchical 20	20	54.68	59.62	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$
Hierarchical 50	50	49.70	57.66	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$
Standard 5	5	59.52	63.93	$\alpha = 1$
Standard 10	10	57.85	63.45	$\alpha = 1$
Standard 20	20	55.41	60.55	$\alpha = 1$
Standard 50	50	50.24	57.47	$\alpha = 1$

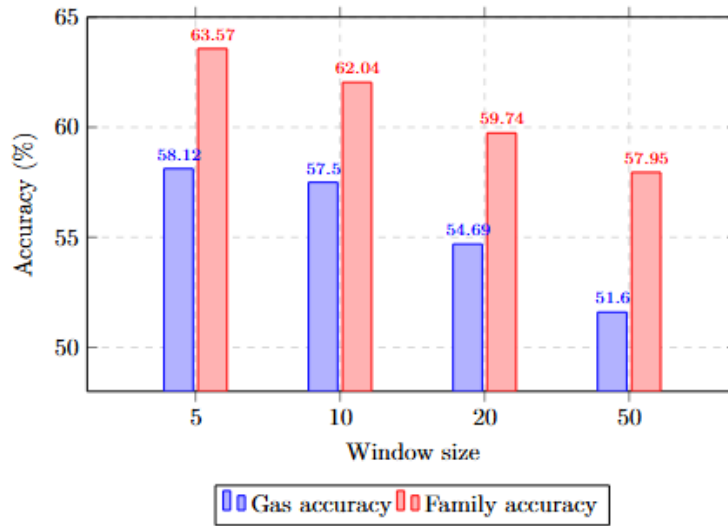


Figure 28: MLP performance with focal loss configuration.

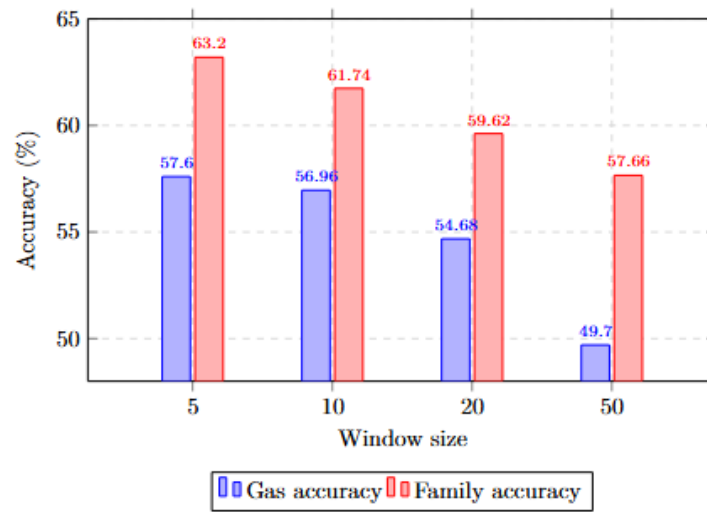


Figure 29: MLP performance with hierarchical loss configuration.

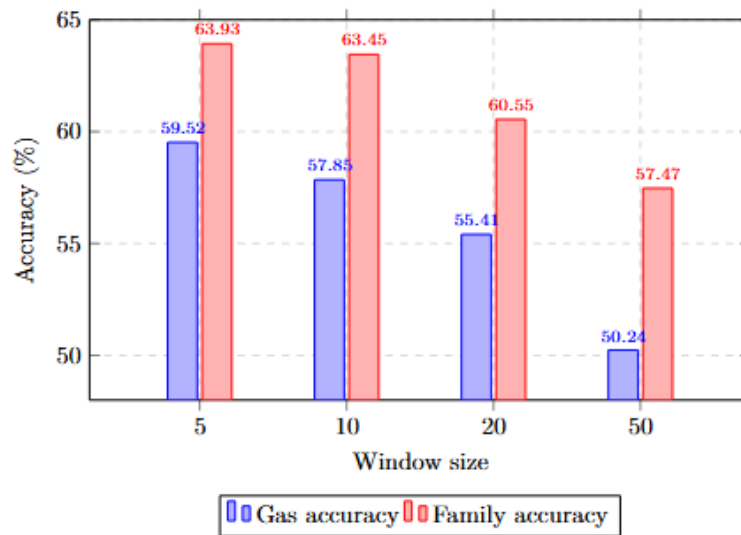


Figure 30: MLP performance with standard loss configuration.

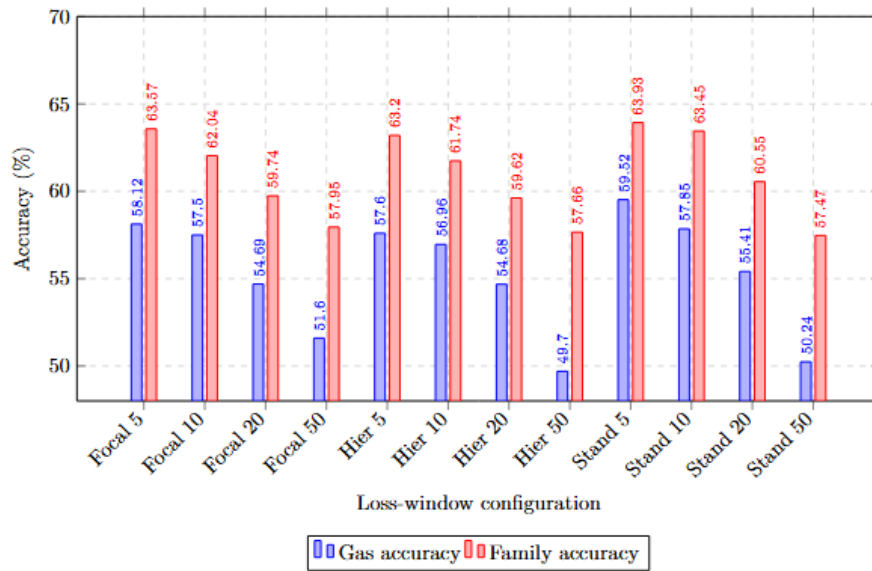


Figure 31: MLP performance for all loss function configurations.

MLP – Standard, Hierarchical and Focal Loss Functions - Statistical Features

The use of statistical features produces a clear improvement compared with raw-feature models. The MLP with statistical features reaches 74.05% gas accuracy, 76.86% family accuracy and 89.56% gas-family consistency. This shows that summarizing the temporal behaviour of the sensor response provides more informative input data than using individual raw sensor rows.

The Random Forest model obtains even higher classification accuracy with statistical features, reaching 79.84% gas accuracy and 80.60% family accuracy. This suggests that tree-based models are particularly effective when the input representation is composed of descriptive statistics. However, the MLP obtains slightly higher consistency, indicating that the neural multi-task structure can better preserve coherence between gas and family predictions.

Table 12: MLP and Random Forest results using statistical features.

Metric	MLP		Random Forest	
	Mean (%)	Std. (%)	Mean (%)	Std. (%)
Gas accuracy	74.05	11.31	79.84	13.68
Family accuracy	76.86	11.42	80.60	11.20
Consistency	89.56	6.59	86.15	4.90

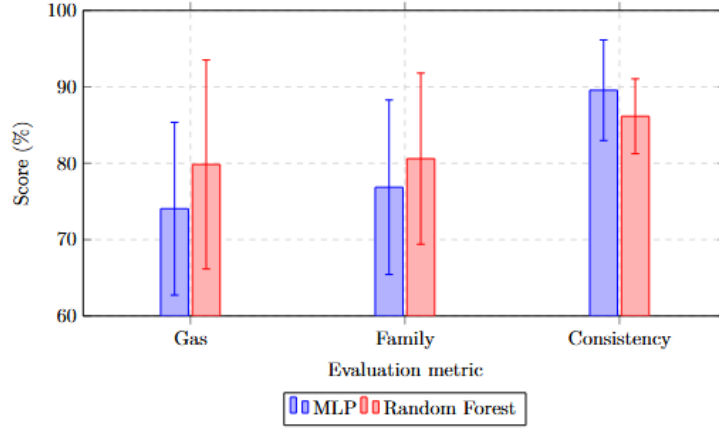


Figure 32: Comparison between MLP and Random Forest using statistical features. Error bars represent the standard deviation across folds.

MLP – Standard, Hierarchical and Focal Loss Functions – Separated Cascade - Raw Features

The separated cascade with raw features obtains results close to the raw multi-task MLP. The gas accuracy remains around the same range as the baseline and the raw multi-task model, while the family accuracy reaches about 64%. This indicates that using the predicted family as an additional input for the gas model does not significantly improve gas classification when the original input is still based on raw features. However, the separated cascade introduces an explicit hierarchical structure between family and gas prediction. This is reflected in the gas-family consistency, which remains high. Therefore, the separated cascade is useful mainly for controlling the coherence of the output, rather than for strongly improving gas accuracy in the raw-feature setting.

Table 13: Separated hard cascade results using raw features.

Metric	Standard	Hierarchical	Focal
Gas mean (%)	58.49	58.64	57.48
Gas std. (%)	7.19	6.97	7.19
Family mean (%)	64.46	64.46	63.63
Family std. (%)	7.00	7.00	6.99
Consistency (%)	88.39	85.44	85.57
Parameters	$\alpha = 1$	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$	$\alpha = 1, \gamma = 2$

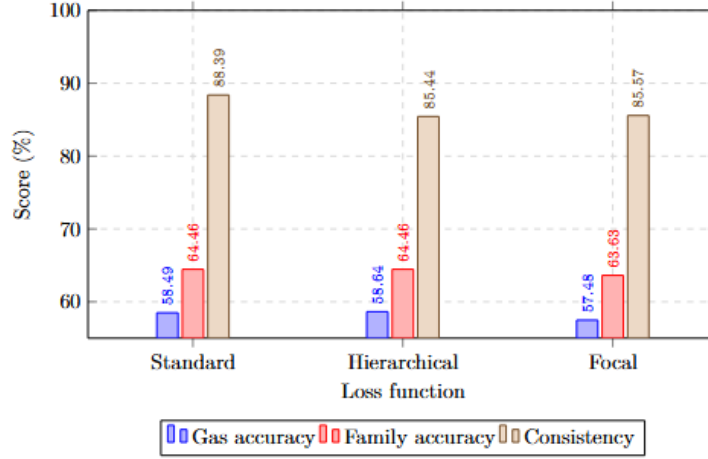


Figure 33: Separated hard cascade performance with raw features.

MLP – Standard, Hierarchical and Focal Loss Functions – Separated Hard Cascade - Raw Features – Rolling Mean Input

The separated hard cascade with rolling mean preprocessing shows lower performance than the corresponding raw-feature cascade. The best gas accuracy is obtained with focal loss, reaching 54.59%, while the best family accuracy is obtained with hierarchical loss, reaching 60.88%. These results confirm that rolling mean preprocessing does not improve the discriminative ability of the model in this configuration. Although smoothing can reduce noise, it may also remove local signal variations that are useful for classifying gases. The consistency remains relatively high, but the reduction in gas and family accuracy suggests that this preprocessing strategy is not optimal for the cascade model.

Table 14: Separated hard cascade results with rolling mean window size 20.

Metric	Standard	Hierarchical	Focal
Gas mean (%)	53.70	52.53	54.59
Gas std. (%)	6.88	7.57	6.59
Family mean (%)	60.45	60.88	60.66
Family std. (%)	7.76	6.72	7.27
Consistency (%)	90.12	87.80	89.03
Parameters	$\alpha = 1$	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$	$\alpha = 1, \gamma = 2$

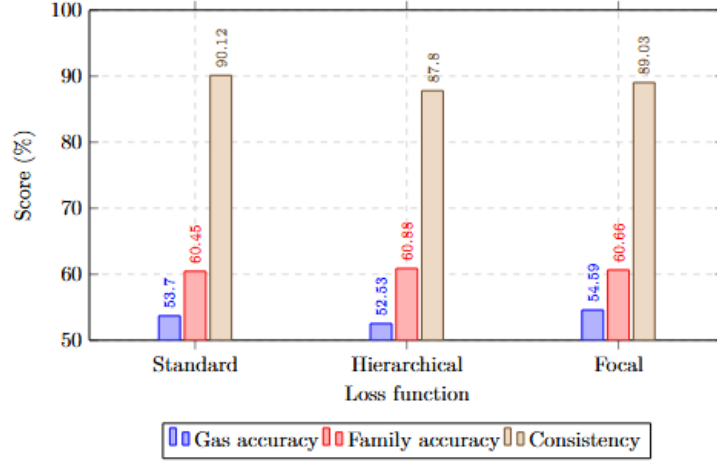


Figure 34: Separated hard cascade performance with rolling mean window size 20.

MLP – Standard, Hierarchical and Focal Loss Functions – Separated Cascade – Statistical Features

The separated cascade with statistical features improves the results compared with the same architecture trained on raw features. With the MLP configuration, the hard representation reaches 65.31% gas accuracy and 71.81% family accuracy. This confirms that statistical descriptors provide more useful information to the cascade model than raw sensor rows.

When Random Forest is used with statistical features, the results are even stronger. The soft Random Forest cascade reaches about 80.39% gas accuracy and 80.36% family accuracy. This result confirms that the combination of statistical descriptors and tree-based classification is particularly effective for this dataset. The improvement is mainly due to the input representation, which captures the temporal behaviour of the acquisition more effectively than raw features.

Table 15: Separated cascade results with hard and soft family representations.

Metric / Parameter	MLP Hard	MLP Soft	RF Hard	RF Soft
Gas mean (%)	65.31	64.31	80.07	80.39
Gas std. (%)	12.42	12.15	13.37	13.53
Family mean (%)	71.81	71.81	80.36	80.36
Family std. (%)	11.17	11.17	11.07	11.07
Family loss weight α	1	1	1	1
Family representation	One-hot	Probability	One-hot	Probability

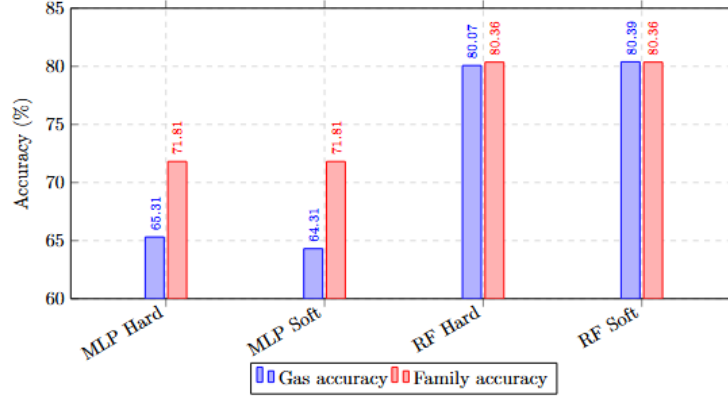


Figure 35: Separated cascade performance with hard and soft family representation for MLP and Random Forest.

MLP – Standard, Hierarchical and Focal Loss Functions – Joint Cascade - Raw Features

The joint cascade with raw features produces results similar to the separated cascade and to the raw multi-task MLP. The standard configuration reaches 58.86% gas accuracy and 64.16% family accuracy. Therefore, the joint training strategy does not provide a clear improvement when the model uses only raw sensor features.

The consistency value remains high, showing that the joint cascade can preserve a coherent relation between gas and family predictions. However, the limited improvement in gas accuracy suggests that the raw representation remains the main bottleneck. The model architecture alone is not sufficient to obtain a strong performance increase without a more informative input representation.

Table 16: Joint hard cascade results using raw features.

Metric	Standard	Hierarchical	Focal
Gas mean (%)	58.86	58.72	58.52
Gas std. (%)	7.12	7.42	7.25
Family mean (%)	64.16	64.16	63.80
Family std. (%)	6.52	6.52	6.19
Consistency (%)	87.64	85.11	85.13
Parameters	$\alpha = 1$	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$	$\alpha = 1, \gamma = 2$

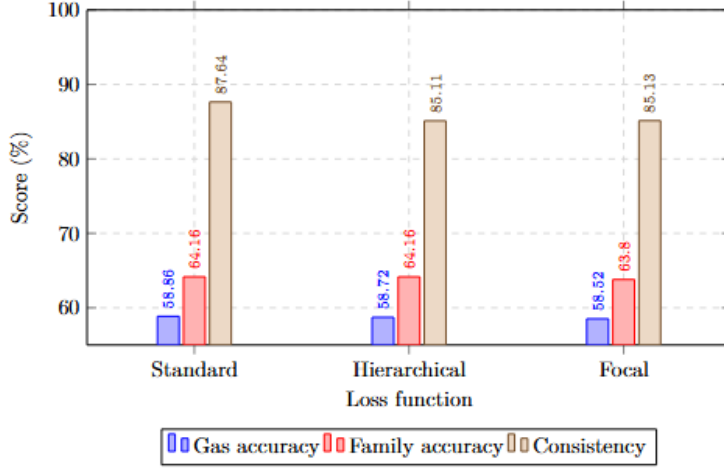


Figure 36: Joint hard cascade performance using raw features.

MLP – Standard, Hierarchical and Focal Loss Functions – Joint Cascade - Raw Features – Rolling Mean Input

The joint cascade with rolling mean window size 20 shows different behaviours for hard and soft family representations. The soft representation generally obtains slightly higher gas accuracy, while the hard representation produces higher consistency values. This indicates that probability-based family information can be useful for gas classification, while one-hot family predictions are more effective in enforcing coherent gas-family outputs.

However, the overall performance remains close to the raw-feature configurations and does not provide a significant improvement. As in the previous rolling-mean experiments, smoothing the signal does not appear to extract enough discriminative temporal information for improving the classification task.

Table 17: Joint cascade results with hard and soft representations using rolling mean window size 20.

Metric	Std. Soft	Std. Hard	Hier. Soft	Hier. Hard	Focal Soft	Focal Hard
Gas mean (%)	57.21	55.91	56.41	55.44	57.34	55.71
Gas std. (%)	7.42	6.79	7.71	6.52	7.80	7.80
Family mean (%)	60.29	62.28	59.48	62.28	60.30	62.01
Family std. (%)	7.67	6.04	7.23	6.04	7.66	7.51
Consistency (%)	77.24	90.19	72.34	87.44	77.50	88.94
Parameters						
Standard	$\alpha = 1$					
Hierarchical	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$					
Focal	$\alpha = 1, \gamma = 2$					

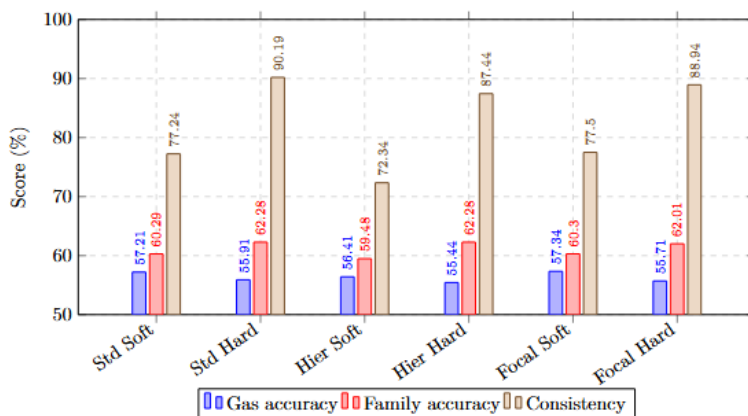


Figure 37: Joint cascade hard and soft performance with rolling mean window size 20.

MLP – Standard, Hierarchical and Focal Loss Functions – Hard Oracle - Raw Features

The hard oracle model obtains the best overall results among the tested configurations. The best configuration is obtained with focal loss, reaching 86.52% gas accuracy, 100% family accuracy and 98.74% gas-family consistency. The family accuracy is equal to 100% by definition, because the true family label is provided directly as input to the gas classifier. This result confirms the importance of family-level information for gas classification. When the correct family is known, the gas classifier can restrict the classification problem to a more coherent chemical context, obtaining a significant improvement over the cascade models. At the same time, the oracle model is not a realistic inference configuration, because the true family would not be available in real deployment. It should therefore be interpreted as an upper-bound reference.

Table 18: Hard oracle results with different loss-function configurations.

Metric	Standard	Hierarchical	Focal
Gas mean (%)	85.83	85.92	86.52
Gas std. (%)	4.06	3.87	3.92
Family mean (%)	100.00	100.00	100.00
Family std. (%)	0.00	0.00	0.00
Consistency (%)	98.48	98.24	98.74
Parameters	$\alpha = 1$	$\alpha = 1, \beta = 0.3, \gamma_{hier} = 3, \lambda_{hier} = 0.5$	$\alpha = 1, \gamma = 2$

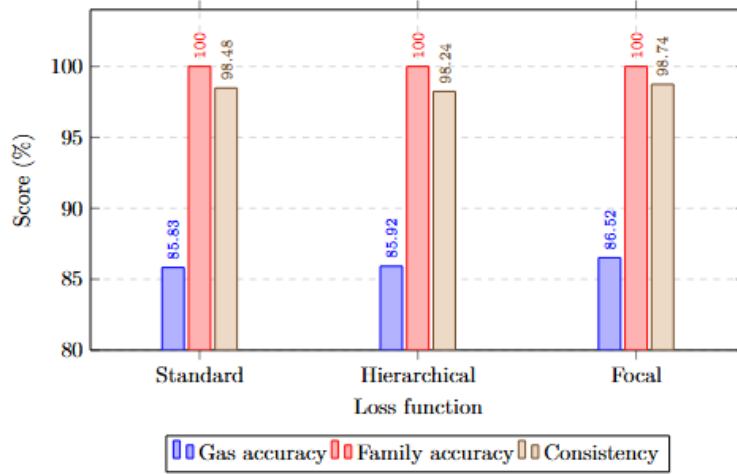


Figure 38: Hard oracle performance with different loss-function configurations.

MLP – Standard Loss Function – Rolling Windows Model - Statistical Features

The window-statistics experiments show that the aggregate-per-file configuration achieves the best gas and family accuracy, with 76.27% and 79.69%, respectively. Among the temporal-window configurations, smaller windows provide better results than larger ones. In particular, window sizes 10 and 20 obtain the highest accuracies among the local-window settings.

The highest consistency among window-based configurations is obtained with window size 10, reaching 90.51%. However, the aggregated representation still provides the best overall trade-off between gas accuracy, family accuracy and consistency. This suggests that summarizing the whole acquisition captures more stable and discriminative patterns than using local temporal windows alone.

Table 19: MLP window-statistics results with standard loss and $\alpha = 1$.

Configuration	Gas acc. (%)	Family acc. (%)	Consistency (%)
Window 10	62.36	67.80	90.51
Window 20	62.15	67.98	90.19
Window 50	61.49	67.70	89.01
Window 70	61.98	67.51	87.03
Window 100	59.45	65.39	86.74
Window 200	59.63	66.23	85.61
Aggregated	76.27	79.69	89.57

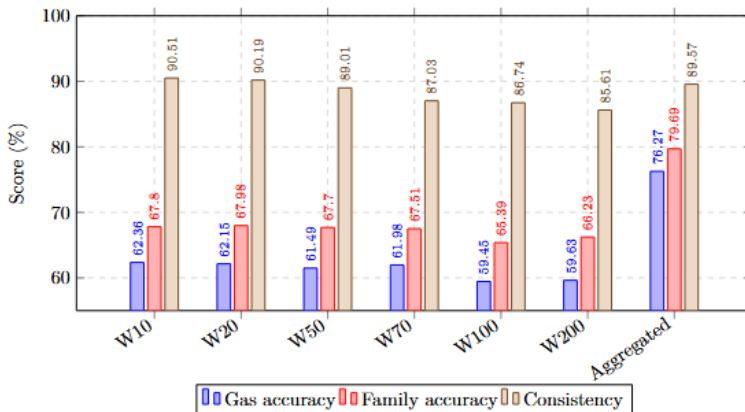


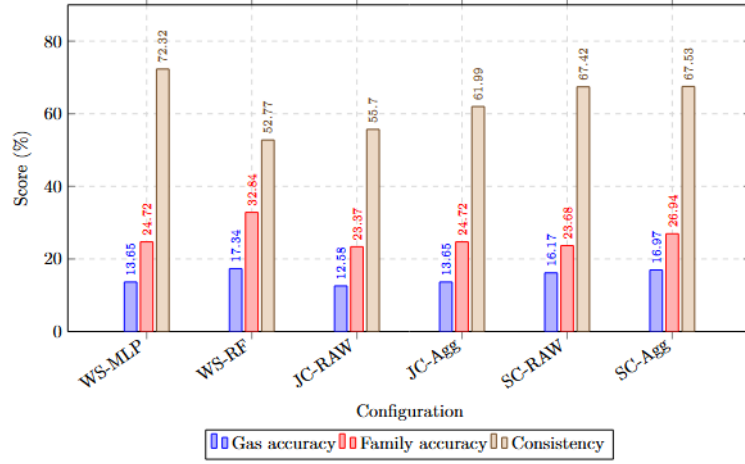
Figure 39: MLP window-statistics performance with standard loss and $\alpha = 1$.

6.3 External Test – Models’ Performances Comparison

The external test results show a strong performance drop for all tested models. The best gas accuracy is obtained by the Random Forest model with statistical features, reaching 17.34%, while the best family accuracy is also obtained by the same model, reaching 32.84%. These values are much lower than those obtained in cross-validation. This difference suggests the presence of a distribution shift between the training data and the external test data. The external acquisitions are likely characterized by more challenging or different experimental conditions, making the trained models less able to generalize. Despite the low gas and family accuracy, some models maintain relatively high gas-family consistency. For example, the statistical MLP reaches a consistency value of about 73.32%, while the separated cascade models remain around 67%. This means that, even when the exact gas prediction is often incorrect, some models still preserve partial chemical coherence between predicted gas and predicted family.

Table 20: External test results with standard loss and $\alpha = 1$.

Configuration	Gas acc. (%)	Family acc. (%)	Consistency (%)
Window stats aggregated per file – MLP	13.65	24.72	72.32
Window stats aggregated per file – RF	17.34	32.84	52.77
Joint cascade – RAW	12.58	23.37	55.70
Joint cascade aggregated per file	13.65	24.72	61.99
Separated cascade – RAW	16.17	23.68	67.42
Separated cascade aggregated per file	16.97	26.94	67.53

Figure 40: External test performance with standard loss and $\alpha = 1$.

6.4 Overall Comparison

In the previous sections of this chapter, the results of the tested models were reported through tables and plots, making the differences between architectures, feature representations and input modalities explicit. The following section provides an overall comparison of these results, highlighting the main trends observed during the experimental evaluation.

Table 21: Overall comparison of the main experimental configurations.

Model configuration	Gas acc. (%)	Family acc. (%)	Consistency (%)
Industrial baseline MLP	58.27	–	–
Raw multi-task MLP	59.83	64.83	–
Raw MLP with rolling mean	59.52	63.93	–
Separated hard cascade, raw features	58.64	64.46	88.39
Joint hard cascade, raw features	58.86	64.16	87.64
MLP with statistical features	74.05	76.86	89.56
RF with statistical features	79.84	80.60	86.15
Separated cascade MLP, statistical features	65.31	71.81	–
Separated cascade RF, statistical features	80.39	80.36	–
MLP window-statistics, aggregated per file	76.27	79.69	89.57
Hard oracle MLP	86.52	100.00	98.74

- The industrial baseline represents the initial reference model. It obtains a mean gas accuracy of 58.27%. Since this model predicts only the gas class, family accuracy and gas-family consistency are not directly evaluated. The baseline is useful as a reference point for understanding whether the proposed extensions improve the original gas classification pipeline.
- The raw multi-task MLP introduces a second output for chemical family prediction. This model slightly improves gas accuracy with respect to the industrial baseline, reaching 59.83%, and provides family accuracy of 64.83%. However, the improvement in gas accuracy is limited. This indicates that adding the family prediction task to the raw-feature MLP is useful for extending the output information, but it does not radically change the classification performance when the input representation remains based on raw sensor rows.
- The rolling-mean configurations do not provide a clear improvement over the raw multi-task MLP. In the best case, the rolling-mean model reaches 59.52% gas accuracy and 63.93% family accuracy. Moreover, increasing the rolling-window size tends to reduce accuracy. This suggests that smoothing the raw signal may remove part of the local information useful for distinguishing gas classes.
- The cascade models based on raw features produce results close to the raw multi-task MLP. The separated hard cascade reaches 58.64% gas accuracy, while the joint hard cascade reaches 58.86%. Therefore, using a predicted family representation as additional input does not significantly improve gas classification when the model still relies on raw features. However, these models introduce an explicit gas-family structure and obtain high consistency values, above 87%.
- A more significant improvement is obtained when statistical features are used. The MLP with statistical features reaches 74.05% gas accuracy and 76.86% family accuracy, while the Random Forest reaches 79.84% gas accuracy and 80.60% family accuracy. These results show that the feature representation has a stronger impact than the architectural modification alone. Statistical descriptors provide a more informative representation of the temporal sensor response than individual raw rows.
- Among the non-oracle models, the best gas accuracy is obtained by the Random Forest with statistical features and by the separated cascade Random Forest configuration. In particular, the separated cascade RF with soft family representation reaches 80.39% gas accuracy and 80.36% family accuracy. This confirms that tree-based models can be highly effective when the input is represented by aggregated statistical descriptors.
- The window-statistics experiments further confirm the importance of the input representation. The aggregate-per-file configuration reaches 76.27% gas accuracy and 79.69% family accuracy, outperforming the temporal-window configurations. Smaller windows, such as window size 10, provide high consistency, but lower gas and family accuracy than the aggregated representation. This suggests that

summarizing the whole acquisition can capture more stable and discriminative patterns than using local temporal windows.

- The hard oracle model achieves the best overall results, with 86.52% gas accuracy, 100% family accuracy and 98.74% consistency. This result is expected, because the correct family label is provided directly to the gas classifier. The oracle configuration does not represent a realistic inference scenario, but it provides an upper-bound reference. The large improvement over the cascade models shows that correct family information is highly useful for gas classification. At the same time, the gap between cascade and oracle results indicates that errors in the predicted family can limit the effectiveness of cascade-based approaches.
- Finally, the external test results show a strong performance drop for all tested configurations. The best gas accuracy on the external test set is obtained by Random Forest with statistical features aggregated per file, reaching only 17.34%. This confirms that the external test set is significantly more challenging and suggests the presence of a distribution shift between the training data and the external test data. Nevertheless, some models maintain relatively high gas-family consistency, indicating that they can still preserve partial hierarchical coherence even when exact gas classification fails.

Overall, the results lead to three main conclusions: statistical feature extraction is the most effective improvement over the industrial baseline, Random Forest performs particularly well when combined with statistical descriptors, while raw-feature neural architectures show only limited improvements, hierarchical and cascade structures are useful for analyzing and improving gas-family coherence, but their effectiveness depends strongly on the quality of the family information provided to the gas classifier.

7. Conclusions and Future Work

Starting from the industrial baseline model, several extensions were implemented and evaluated, including multi-task models, cascade architectures, oracle configurations and models based on statistical feature extraction. The work highlights both strengths and limitations of the explored approaches. On the positive side, the study shows that it is possible to improve the modeling of sensor data by considering richer representations and by introducing structured prediction strategies. In particular, the use of hierarchical reasoning and alternative feature representations provides useful insights into how the classification task can be approached beyond the baseline solution. At the same time, some limitations remain. Certain approaches introduce additional complexity, either in terms of computational cost or model design, which may limit their applicability in constrained environments. Moreover, ensuring consistency between different levels of prediction, such as gas and chemical family, remains a challenging aspect that is not fully resolved by the current methods. Finding a balance between simpler models, easy to deploy and maintain, and more complex ones, that can offer advantages but requiring careful optimization, is the true goal to be achieved.

Future work should therefore focus on improving this balance. Possible directions include reducing model complexity while preserving informative representations, exploring more efficient feature extraction techniques and refining hierarchical classification strategies to enforce stronger consistency between predictions. In this context, the oracle model can serve as a useful upper bound, providing a reference for the maximum achievable performance when family information is perfectly known and guiding the design of more effective cascade architectures. Additionally, further efforts are needed to improve robustness and generalization, for example by considering more diverse data conditions and by developing methods that can better handle variability in sensor responses.

8. Bibliography

- [1] WHO, *Ambient (outdoor) air pollution*, fact sheet.
- [2] Fine et al., *Metal Oxide Semi-Conductor Gas Sensors in Environmental Monitoring*, Sensors, 2010.
- [3] Capelli et al., *Electronic Noses for Environmental Monitoring Applications*, Sensors, 2014.
- [4] Wilson and Baietto, *Applications and Advances in Electronic-Nose Technologies*, Sensors, 2009.
- [5] Wikipedia contributors, *Gas detector*, Wikipedia, The Free Encyclopedia.
- [6] *About*, Sensichips s.r.l. official page
- [7] V. Nasteski, *An overview of the supervised machine learning methods*, Faculty of Information and Communication Technologies, Partizanska bb, Macedonia.
- [8] J. Boedecker, M. Riedmiller, *Multi-Layer Perceptrons*, Machine Learning Lab, University of Freiburg.
- [9] F. Patrizi, *ML lecture notes*, ML course, Sapienza University of Rome
- [10] *RandomForestClassifier*, scikit-learn 1.9.0 documentation
- [11] L. Breiman, *Random Forests*, Statistics Department University of California, Berkley, 2001
- [12] Y. Zhang and Q. Yang, *An overview of multi-task learning*, National Science Review, vol. 5, no. 1, pp. 30–43, 2018
- [13] F. Chahkoutahi, M. Khashei, and N. Molaverdi, *Loss functions in classification: A comprehensive overview and comparative study*, Applied Soft Computing, vol. 184, 113778, 2025

- [14] J. Lang, Z. Zeng, T. Ma, S. He, *Leaking Gas Source Tracking for Multiple Chemical Parks within An Urban City*, Centre for Optical and Electromagnetic Research, Zhejiang Provincial Key Laboratory for Sensing Technologies, College of Optical Science and Engineering, Zhejiang University, Hangzhou 310058, China, 2023
- [15] V. W. Lumumba, D. Kiprotich, M. L. Mpaine, N. G. Makena, M. D. Kavita, *Comparative Analysis of Cross-Validation Techniques: LOOCV, K-folds Cross-Validation, and Repeated K-folds Cross-Validation in Machine Learning Models*, Department of Physical Science, Chuka University, Chuka, Kenya, 2024
- [16] S. Raschka, J. Patterson, and C. Nolet, *Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence*, Information, vol. 11, no. 4, 193, 2020.
- [17] Pandas development team, *pandas.read_csv*, pandas API reference documentation.
- [18] NumPy Developers, *NumPy documentation: ndarray and array operations*, NumPy documentation.
- [19] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [20] Scikit-learn Developers, *Scikit-learn API reference: preprocessing, metrics and RandomForestClassifier*, Scikit-learn documentation.
- [21] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Advances in Neural Information Processing Systems, 2019.
- [22] PyTorch Developers, *torch.utils.data: Dataset and DataLoader*, PyTorch documentation.
- [23] PyTorch Developers, *torch.optim and training with PyTorch*, PyTorch documentation.
- [24] Python Software Foundation, *argparse — Parser for command-line options, arguments and subcommands*, Python documentation.
- [25] Python Software Foundation, *The Python Standard Library: json and pathlib*, Python documentation.

- [26] Matplotlib Developers, *matplotlib.pyplot documentation*, Matplotlib documentation.
- [27] Seaborn Developers, *seaborn.heatmap*, Seaborn documentation.
- [28] A. Shivanna, J. Gounley, H. A. Hanson, A. Spannaus, *Multi-Label Classification with Constraint-Based Learning for Hierarchical Consistency*, 2025 IEEE Conference on Artificial Intelligence (CAI)